Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Ecole Polytechnique Internationale Privée de Tunis

Final Year Project

For the award of the degree of

Engineering Degree in Computer Science, Networks and Multimedia
Specialization: Information Systems and Software Engineering

Presented and defended by:

Emna CHEBBI

30 June 2026

Design and Implementation of a DevSecOps and GitOps Approach for an AI-Driven Multichannel Web Platform

Prepared at :

BeBouncy

JURY

Ms.	Imen Merdassi	Chairperson
Mr.	Haythem Ghazouani	Reviewer
Mr.	Feker Skandrani	Industrial Supervisor
Mr.	Houcemeddine Hermassi	Academic Supervisor

FOR.29|V.01

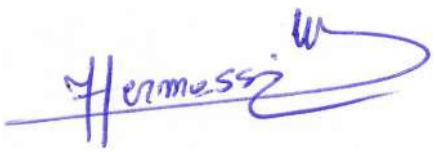
Année Universitaire : **2025/2026**

Industry Supervisor, **Mr. Feker Skandrani**

Signature and Stamp

Academic Supervisor, **Mr. Houcemeddine Hermassi**

Signature



Dedication

To my beloved parents,

Your love, guidance, and unwavering support have been the foundation of everything I have achieved. To my mother, whose patience, kindness, and strength inspire me every day, and to my father, whose encouragement, values, and constant belief in me have shaped my determination and perseverance. This work is a reflection of the principles you instilled in me and the sacrifices you made for my success.

To my sisters,

For your continuous support, affection, and presence throughout my journey. Your encouragement has been a constant source of motivation.

To my friends,

For your support, encouragement, and the meaningful moments we shared. You have helped me maintain balance, motivation, and resilience during challenging times.

And to all those who, directly or indirectly, contributed to this project,

Your support, encouragement, and involvement have been truly invaluable.

I dedicate this work to you all.

Emna Chebbi

Acknowledgements

First and foremost, I express my sincere gratitude to Almighty God for His guidance and support, which enabled the successful completion of this work.

I would like to extend my deepest appreciation to all those who contributed, directly or indirectly, to the successful completion of this internship under the best possible conditions.

I sincerely thank the members of the jury for taking the time to evaluate this work.

I would like to express my profound gratitude to my academic supervisor, Mr. Houcemeddine Hermassi, for his availability, guidance, and valuable advice throughout this project. His support and insightful recommendations were essential in shaping the success of this work.

I also extend my heartfelt thanks to my industrial supervisors, Mr. Feker Skandrani and Mr. Khalil Turki, for their guidance, patience, professionalism, and continuous support throughout this internship. Their trust and mentorship allowed me to successfully carry out this project.

I am also grateful to the entire BeBouncy team for their warm welcome, assistance, cooperation, and kindness throughout the internship period.

Finally, I would like to express my sincere appreciation to all the professors and academic staff at the Private International Polytechnic School of Tunis for their education, guidance, and continuous support throughout my academic journey.

Contents

List of Figures

List of Tables

List of Abbreviations

AI	=	Artificial Intelligence
API	=	Application Programming Interface
AlertManager	=	Prometheus Alert Routing and Notification System
ArgoCD	=	Argo Continuous Delivery
BI	=	Business Intelligence
CI/CD	=	Continuous Integration and Continuous Deployment
CIS	=	Center for Internet Security
CISA	=	Cybersecurity and Infrastructure Security Agency
CLI	=	Command-Line Interface
CNCF	=	Cloud Native Computing Foundation
CPU	=	Central Processing Unit
CRD	=	Custom Resource Definition (Kubernetes API Extension Object)
CRI	=	Container Runtime Interface
CRM	=	Customer Relationship Management
CVE	=	Common Vulnerabilities and Exposures
Checkov	=	Infrastructure-as-Code Security Scanner
CycloneDX	=	Software Bill of Materials Standard Format
DAST	=	Dynamic Application Security Testing
DNS	=	Domain Name System
DevOps	=	Development and Operations

DevSecOps	=	Development, Security, and Operations
GHCR	=	GitHub Container Registry
Git	=	Distributed Version Control System
GitOps	=	Git-based Operations and Deployment Methodology
Gitleaks	=	Git-Based Secrets Scanning Tool
Grafana	=	Open-source Analytics and Visualisation Platform
HPA	=	Horizontal Pod Autoscaler
HTTP	=	HyperText Transfer Protocol
HTTPS	=	HyperText Transfer Protocol Secure
HashiCorp Vault	=	Identity-Based Secrets Management Platform
Helm	=	Kubernetes Package Manager
ICT	=	Information and Communication Technologies
IaC	=	Infrastructure as Code
Ingress	=	Kubernetes API Object managing external access to Services
JSON	=	JavaScript Object Notation (Data Interchange Format)
K3s	=	Lightweight Kubernetes Distribution
K8s	=	Kubernetes (abbreviated form)
Kube-State-Metrics	=	Kubernetes State Metrics Exporter for Cluster Objects
Kubernetes	=	Open-source Container Orchestration System
Metrics	=	Quantitative Performance and Health Measurements
NSA	=	National Security Agency
NVD	=	National Vulnerability Database

Node	=	Worker Machine in a Kubernetes Cluster
Node Exporter	=	Prometheus Metrics Exporter for Host-level System Metrics
NodePort	=	Kubernetes Service Type exposing a Port on each Node
OIDC	=	OpenID Connect
OS	=	Operating System
OSV	=	Open Source Vulnerabilities Database
OVH	=	On Vous Héberge (Cloud Hosting Provider)
OWASP ZAP	=	Open Worldwide Application Security Project Zed Attack Proxy
PM2	=	Process Manager 2
PR	=	Pull Request
PVC	=	Persistent Volume Claim (Kubernetes Storage Resource)
Pod	=	Smallest Deployable Unit in Kubernetes
PodMonitor	=	Prometheus Operator Resource for Monitoring Kubernetes Pods
PromQL	=	Prometheus Query Language
Prometheus	=	Open-source Monitoring and Alerting Toolkit
PrometheusRule	=	Kubernetes Custom Resource for Defining Prometheus Alerting Rules
QA	=	Quality Assurance
RAM	=	Random Access Memory
RBAC	=	Role-Based Access Control
ROI	=	Return on Investment
SARL	=	Société à Responsabilité Limitée

SAST	=	Static Application Security Testing
SBOM	=	Software Bill of Materials
SCA	=	Software Composition Analysis
SHA	=	Secure Hash Algorithm
SMEs	=	Small and Medium-sized Enterprises
SMM	=	Social Media Management
SMTP	=	Simple Mail Transfer Protocol
SQL	=	Structured Query Language
SSH	=	Secure Shell
SVN	=	Apache Subversion (Version Control System)
SaaS	=	Software as a Service
ServiceMonitor	=	Prometheus Operator Resource for Defining Metric Scraping Targets
SonarCloud	=	Cloud-Based Static Code Analysis Platform
SonarQube	=	Static Code Quality and Security Analysis Platform
TLS	=	Transport Layer Security
TSDB	=	Time Series Database
TTL	=	Time To Live
Trivy	=	Open-source Vulnerability and Security Scanner
UI	=	User Interface
VM	=	Virtual Machine
YAML	=	YAML Ain't Markup Language

cAdvisor	=	Container Advisor - Container Resource Usage Metrics Exporter
env	=	Environment (Configuration File)
etcd	=	Distributed Key-Value Store (Kubernetes backing store)
npm	=	Node Package Manager
yml	=	YAML File Extension

General Introduction

In recent years, information technology has undergone rapid transformation, reshaping how organizations design, deploy, and manage digital services. Digital communication channels such as Facebook, WhatsApp, Instagram, and LinkedIn have become essential pillars of interaction, generating massive volumes of data — messages, comments, and engagement signals — that require advanced processing to extract meaningful insights.

In this context, artificial intelligence has emerged as a key enabler for modern digital platforms. Techniques such as semantic analysis, sentiment detection, and predictive analytics allow organizations to better understand user behavior, detect harmful content, and optimize communication strategies in real time. However, building and operating such intelligent systems at scale introduces significant technical challenges, including application reliability, system security, continuous updates, and compliance with quality standards.

To overcome these limitations, modern practices such as DevOps and DevSecOps have been widely adopted. DevOps emphasizes collaboration between development and operations teams through Continuous Integration and Continuous Deployment (CI/CD) pipelines, while DevSecOps extends this by embedding security throughout the entire software lifecycle. Complementing these practices, containerization technologies like Docker enable lightweight, portable execution environments, and observability tools provide real-time visibility into system performance for proactive anomaly detection.

In this context, BeBouncy provides an innovative platform designed to centralize and optimize multichannel communication for organizations, unifying social media channels and integrating AI capabilities for intelligent digital interaction management.

The objective of this final year project is to design and implement a DevSecOps approach for this AI-powered multichannel web application, encompassing a secure and automated delivery pipeline, containerization, CI/CD, security analysis, secrets management, and system observability.

Chapter 1

Project Context and General Overview

Contents

Introduction

The success of any project strongly depends on a clear understanding of its initial context. This chapter presents the general framework of our internship project carried out at BeBouncy. It begins with an introduction to the host organization, its mission, and its positioning within the social media management landscape. The chapter then outlines the project context and problem statement, detailing the technical challenges that motivated this work. A competitive analysis is conducted to situate BeBouncy within both the international and Tunisian markets, followed by an in-depth examination of the existing system and its limitations. A DevSecOps-based solution is then proposed in response to these gaps. Building on that foundation, the chapter proceeds to the formal identification of functional and non-functional requirements, a detailed design of the proposed system architecture across its pipeline, logical, and physical layers, and the development methodology selected to govern the project. The chapter closes with the sprint planning schedule and an overview of the technology stack adopted throughout the implementation.

1.1 Presentation of the Host Organization

1.1.1 BeBouncy

BeBouncy is an innovative startup specialized in Information and Communication Technologies (ICT), with a focus on artificial-intelligence-driven social media management solutions. Founded in May 2019 and supported by the French Tech ecosystem, specifically the Grand Paris initiative, BeBouncy provides an all-in-one SaaS platform that enables companies, agencies, and individual professionals to efficiently manage, analyze, and optimize their presence across multiple social networks including Facebook, Instagram, and LinkedIn, with TikTok integration on the roadmap.

At the heart of the platform stands **Nimbus**, BeBouncy's proprietary AI agent. Nimbus goes beyond simple automation: it analyzes content performance, monitors audience sentiment, suggests optimal posting times, and even proposes or auto-generates replies to community interactions.

Figure 1.1: BeBouncy Logo

1.1.2 Mission of BeBouncy

BeBouncy's mission is built around three core pillars: simplification, intelligence, and growth. More precisely, the platform aims to:

- Offer companies and professionals an innovative and high-performance platform for piloting their social networks, equipped with user-friendly tools, sophisticated features, and high-quality support.
- Help its clients plan, publish, monitor, and analyze their content across multiple social platforms, reducing the time spent on manual content management, currently estimated at more than ten hours per week for a typical business.
- Facilitate the centralized management of all digital activities through a unified dashboard, eliminating the need to juggle multiple disconnected tools.
- Strengthen online visibility, foster audience engagement, and support the growth and success of businesses within the digital ecosystem with measurable outcomes such as up to 85% better audience understanding, 60% more impact through personalized strategies, and 75% time savings through automated posting.

These missions are concretely reflected in the platform's four main feature modules: **Schedule** (smart content calendar), **Analyse** (performance analytics and automated reports), **Moderate** (AI-assisted community moderation), and **Gamification** (team performance tracking with reward mechanics).

1.1.3 Company Overview

Attribute	Details
Company Name	BeBouncy
Sector	Information and Communication Technologies (ICT)
CEO	Assad Ben Mrad
Legal Form	SARL
Founded	May 2019
Ecosystem	French Tech / Grand Paris
Target Networks	Facebook, Instagram, LinkedIn (TikTok coming soon)
Platform Website	https://beta.bebouncy.com/
Showcase Website	https://www.bebouncy.com/

Table 1.1: BeBouncy Company Overview

1.2 Project Presentation

1.2.1 Project Context

With the rapid evolution of modern web applications, companies are increasingly required to adopt efficient development and deployment practices. The pressure to deliver new features quickly, maintain system reliability, and ensure security has given rise to the DevSecOps movement, a culture and set of practices that integrates Development, Security, and Operations into a single, continuous workflow.

At BeBouncy, the platform has grown functionally over the years, offering advanced features for social media management. However, the underlying technical infrastructure has not kept pace with this functional growth. Deployments are performed manually over SSH, environment configurations diverge between the QA and production servers, and security checks are conducted only at the end of the development cycle, if at all. The absence of automated pipelines, container orchestration, and proper observability tools creates a fragile foundation that hinders the team's ability to scale and iterate rapidly.

This internship project was initiated precisely to address these gaps. The objective is to design and implement a modern DevSecOps infrastructure that automates deployments, enforces environment consistency, integrates security early in the pipeline, and provides comprehensive observability over all running services.

1.2.2 Problem Statement

BeBouncy's platform is built to help its users manage their digital presence more efficiently, yet internally the engineering team faces analogous inefficiencies in its own operational processes. As the platform has grown, the gap between what the product promises its customers and how the product itself is built and delivered has become a concrete technical liability.

Concretely, the current system presents the following critical challenges:

- **Slow and error-prone deployment processes:** Each deployment requires a developer to manually SSH into the production server, pull the latest code, run installation commands, and restart services using PM2. This process is repetitive, not reproducible, and highly susceptible to human error.
- **Inconsistency between environments:** The QA and production servers have diverged over time in terms of Node.js versions, installed packages, and OS configurations.
- **Limited observability:** There is no centralized monitoring or alerting system. When a service degrades or crashes, the team learns about it from user reports or logs rather than from automated alerts.
- **Late integration of security mechanisms:** Vulnerability scanning and dependency audits, when performed at all, occur as an afterthought late in the development cycle. This increases the exposure window for exploitable weaknesses.
- **Absence of scalability mechanisms:** Scaling a service requires manually adjusting PM2 cluster instances and restarting processes, with no automatic response to load variations.

1.3 Competitive Analysis

The social media management (SMM) software market is highly competitive at the international level, with several well-established players offering overlapping feature sets. In Tunisia, however, no direct SaaS competitor occupies the same space, which represents a significant market opportunity for BeBouncy. The following comparative study examines the major international platforms alongside BeBouncy, identifying shared features, differentiating factors, and the limitations of each solution.

1.3.1 Existing Solutions and Their Limitations

Table 1.2 presents the main competing platforms, their core functionality, and their known limitations. This analysis is based on publicly available information from each platform's official website.

Platform	Description	Limitations
 Figure 1.2: Hootsuite Logo	Hootsuite is a long-established social media management tool that enables scheduling, publishing, monitoring, analytics, and team collaboration across major platforms like Facebook, Instagram, LinkedIn, X, YouTube, and Pinterest.	However, its advanced features (reporting, social listening, ROI tracking) are expensive, and the interface can feel complex for beginners and smaller teams.
 Figure 1.3: Buffer Logo	Buffer is a simple, clean social media scheduling tool used mainly by individuals and small teams to plan and publish posts on platforms like Facebook, Instagram, LinkedIn, and X.	However, it has limited analytics and lacks advanced features such as social listening, CRM integration, and strong team collaboration, making it less suitable for larger organizations.
 Figure 1.4: Sprout Social Logo	Sprout Social is an enterprise-grade platform offering publishing, engagement, analytics, social listening, and CRM integration. It is oriented toward large organizations and agencies that require deep reporting and cross-team workflows.	Pricing is among the highest in the market, placing it out of reach for startups and SMEs. Some advanced features also require a significant learning curve, and onboarding new team members can be time-consuming.

(continued on next page)

Platform	Description	Limitations
 Figure 1.5: Later Logo	Later focuses on visual content planning with a drag-and-drop calendar and link-in-bio tool, making it popular for Instagram and TikTok users like influencers, e-commerce brands, and creative agencies.	However, it is less suited for broader use since support for platforms like LinkedIn and X is limited, and its analytics and engagement features are relatively basic.
 Figure 1.6: Metricool Logo	Metricool is an all-in-one tool that combines scheduling, analytics, and competitive benchmarking across multiple social and advertising platforms, offering detailed performance insights like reach and engagement.	However, its interface can feel cluttered when handling many accounts, and it lacks some niche integrations, with customer support sometimes considered slow to respond.
 Figure 1.7: Agorapulse Logo	Agorapulse is a full-featured tool that offers publishing, a unified social inbox, CRM-style contact management, and detailed reporting with ROI tracking, making it well-suited for agencies and marketing teams managing multiple clients.	However, its cost rises quickly with more users or accounts, and some users report occasional sync delays with connected social platforms.

Table 1.2: International Competitors of BeBouncy

1.3.2 Market Positioning, Core Features, and Differentiation

The social media management (SMM) market is defined by a stable set of core functionalities that have become industry standards across all major platforms. These include content scheduling, multi-platform publishing, analytics and reporting, engagement management, centralized dashboards, performance optimization, and team collaboration tools. BeBouncy implements all of these essential capabilities,

ensuring compatibility with established international solutions while providing a unified and efficient user experience.

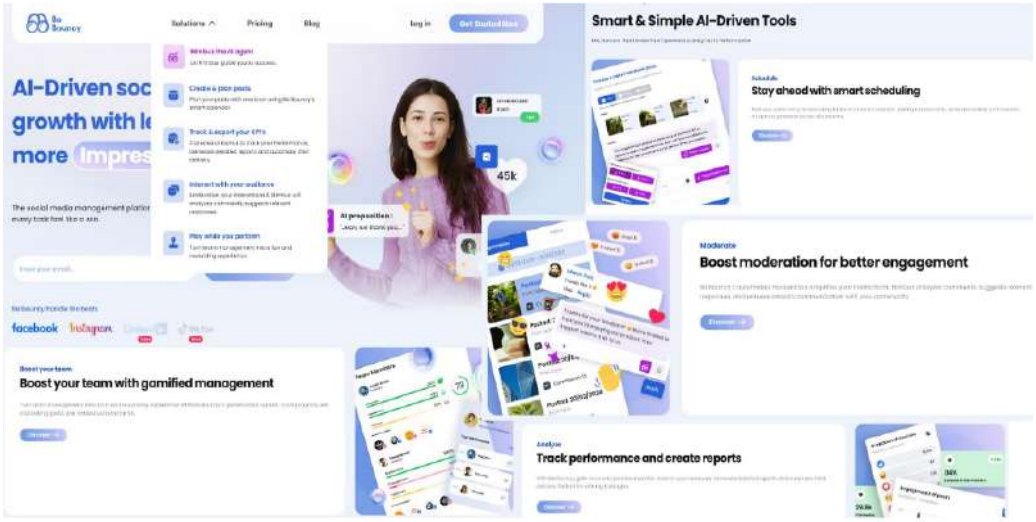


Figure 1.8: Overview of BeBouncy Features

Beyond these baseline features, BeBouncy introduces several key differentiators that enhance its strategic value. As illustrated in Table 1.3, the platform integrates an advanced AI agent (Nimbus), intelligent conversation automation, a gamification system, and an extended multi-channel scope combining social media, messaging, and CRM functionalities. It also positions itself as a more accessible solution for SMEs, agencies, and individual creators compared to enterprise-focused competitors.

Aspect	BeBouncy	Competitors
AI Agent	Nimbus: advanced AI for content analysis, strategic recommendations, and auto-reply suggestions	Limited AI support, mostly restricted to basic caption or timing suggestions
Conversation Automation	Context-aware automation using sentiment analysis and intelligent response generation	Mainly manual inbox management with limited rule-based automation
Gamification System	Performance tracking with goals, badges, and achievement rewards	Not available in major competing platforms
Multi-channel Scope	Social networks, messaging platforms, and CRM-style contact management	Primarily limited to social media platforms only
Market Positioning	First structured SaaS SMM platform in Tunisia with regional focus	International platforms with no local market specialization
Accessibility	Affordable and flexible pricing for SMEs and creators	High-cost enterprise-oriented solutions

Table 1.3: Differentiation Factors of BeBouncy vs. Competitors

A review of the Tunisian market further highlights a significant gap: there are currently no local SaaS competitors in the social media management domain. As a result, Tunisian businesses rely entirely on international platforms, which often lack localized support in terms of language, pricing accessibility, and regional compliance considerations. This situation positions BeBouncy as a first-mover opportunity within Tunisia and potentially the broader North African region, addressing unmet local needs while competing with established global players.

1.4 Existing System Analysis

1.4.1 Current Architecture Overview

Before this project, BeBouncy's infrastructure consisted of four Ubuntu 24.04 virtual machines hosted at OVHcloud, managed entirely through direct SSH access and the PM2 process manager. The architecture followed a quasi-monolithic deployment model: each service was deployed as a Node.js process running directly on the operating system, without any containerization or isolation mechanism.

The four machines are organized as follows:

- **Production Machine 1 — API & AI:** Hosts the main backend API server (`api-master`) alongside the AI service and its associated cron jobs (`ai-project`, `ai-project-crons`).
- **Production Machine 2 — Web Front-End:** Dedicated to the Next.js web application that serves the user-facing dashboard.
- **Production Machine 3 — Backoffice & Microservices:** Runs the administration back-office and the Agenda-based microservices layer, including a scheduler (`agenda-scheduler`) and multiple cluster workers (`agenda-worker`).
- **QA Machine:** A single server that consolidates all four services (API, Web, AI, and Microservices) for quality assurance testing prior to production deployment.

All processes on these machines are managed by PM2, a Node.js process manager that provides basic clustering, auto-restart on crash, and log rotation. A typical PM2 process list on the QA machine appears as follows:

id	name	mode	↺	status	cpu	memory
301	agenda-scheduler	fork	1	online	0%	146.0mb
302	agenda-worker	cluster	0	online	0%	129.0mb
303	agenda-worker	cluster	0	online	0%	129.4mb
304	agenda-worker	cluster	0	online	0%	129.9mb
305	agenda-worker	cluster	0	online	0%	126.6mb
641	ai-project	fork	1	online	0%	90.6mb
642	ai-project-crons	fork	1	online	0%	91.4mb
634	api-master	fork	1	online	0%	140.5mb
567	b3ynd-qa	fork	37	online	0%	42.0mb
643	npm	fork	1	online	0%	69.6mb

Figure 1.9: PM2 Process Status

1.4.2 Current Deployment Workflow

The deployment process follows a fully manual, sequential procedure that requires direct server access at every step. It can be summarized as follows:

Development: Developers work on feature branches in Git repositories. Merges to the main branch are performed manually, without automated branch protection rules or required code reviews. There is no gate to prevent broken code from being merged.

Build: Once a merge is complete, the developer SSH-es into the target server and manually pulls the latest code. Dependencies are installed by running `npm install`, and the TypeScript source is compiled by running `npm run build`. There is no standardized build environment, meaning the Node.js version on the server may differ from the developer's local machine.

Deployment: The updated process is restarted using PM2 (`pm2 restart api-master`, for example). If the new build contains an error, the process either crashes immediately at which point PM2 restarts it indefinitely, masking the failure or it starts but behaves incorrectly in production.

Security: Security checks are not integrated into the development or deployment workflow. Dependency vulnerability audits and secret scanning are not performed automatically, leaving the codebase exposed to undetected vulnerabilities for extended periods.

Monitoring: There is no centralized monitoring dashboard. Developers check service health by consulting PM2 logs manually, or they are alerted by users reporting errors. There are no automated alerts for CPU spikes, memory saturation, or service unavailability.

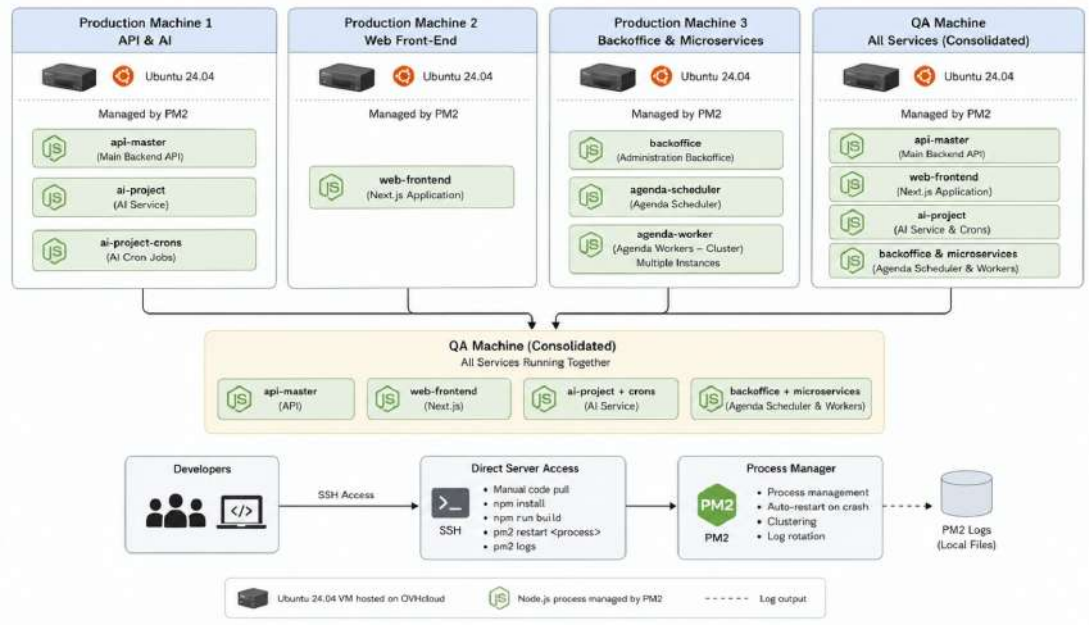


Figure 1.10: Current System Architecture

1.4.3 Observed Limitations and Their Impact

The current infrastructure, while functional in the early stages of the startup, now presents a growing set of critical limitations as the platform scales:

- **No CI/CD pipelines:** Every deployment is a manual, undocumented procedure. There is no

automated testing gate before code reaches production. This increases the risk of regressions and places a high cognitive burden on developers, who must remember and execute a sequence of commands correctly every time.

- **Manual and fragile deployment:** Some processes on the QA machine show 37 restarts in PM2, a clear indicator of instability that has gone unaddressed due to the lack of alerting. Two zombie processes were also observed on the QA server, indicating resource leaks that are not being cleaned up automatically.
- **No containerization:** Services share the same OS environment without isolation. A dependency conflict between services, or a runaway process consuming excessive CPU or memory, can destabilize the entire machine. There is no resource limit enforcement.
- **Environment drift and inconsistency:** The QA and production servers have diverged over time through accumulated manual changes. Bugs that reproduce only in production but not in QA are frequently caused by these untracked environmental differences.
- **No scalability mechanism:** Handling increased traffic requires manually editing PM2 configuration files and restarting processes. There is no horizontal autoscaling, no load balancing at the application layer, and no mechanism for gracefully handling traffic spikes.
- **Late and incomplete security integration:** Secrets are stored in `.env` files distributed across multiple repositories. There is no centralized secret management, no automated scanning for exposed credentials or vulnerable dependencies, and no enforced security policy at the pipeline level.

1.5 Proposed Solution

In response to the limitations identified above, this internship project proposes the design and implementation of a complete DevSecOps infrastructure for BeBouncy, built around the following core principles: automation, immutability, security-by-default, and observability.

Containerization with Docker: Each service (API, Web, AI, Microservices) is packaged into a Docker container using a multi-stage Dockerfile. This eliminates environment drift by guaranteeing that the same image runs identically in QA and production. Production images are lean (150–250 MB) and built without development-only dependencies.

Container Orchestration with Kubernetes: The four services are deployed into Kubernetes clusters: a single-node cluster for QA and a three-node cluster for production. Kubernetes brings

automatic pod rescheduling on failure, horizontal scaling, rolling updates with zero downtime, and built-in service discovery via DNS, replacing the fragile PM2-based process management.

CI/CD Pipelines with GitHub Actions: Automated pipelines are triggered on every code push to application repositories. Each pipeline run executes a security scan of the source code, builds and tests the Docker image, pushes it to the GitHub Container Registry (GHCR), and automatically updates the infrastructure repository with the new image tag.

GitOps with ArgoCD: The BB-INFRA repository serves as the single source of truth for all Kubernetes deployments. ArgoCD continuously monitors this repository and synchronizes the cluster state to match. No deployment ever happens through manual SSH; every change is a reviewed, auditable Git commit.

Infrastructure as Code: All Kubernetes resources are defined through Helm chart templates parameterized by environment-specific values files. Configuration drift becomes impossible; the cluster state is fully deterministic and reproducible from the Git repository alone.

Integrated Security: Security is integrated directly into the CI/CD pipeline using a shift-left DevSecOps approach. **Trivy** scans source code and container images for vulnerabilities and exposed secrets, **SonarQube Cloud** performs static code analysis, **Checkov** validates Infrastructure as Code configurations, and **OWASP ZAP** executes dynamic security testing on deployed applications. Secrets are managed securely using **Vault** and **Kubernetes Secrets**, avoiding plaintext storage in repository files.

Observability with Prometheus and Grafana: A monitoring stack is deployed alongside the application services, collecting metrics from the Kubernetes nodes and application containers. Grafana dashboards provide real-time visibility into resource usage, request rates, and error rates, with alerting rules that notify the team proactively.

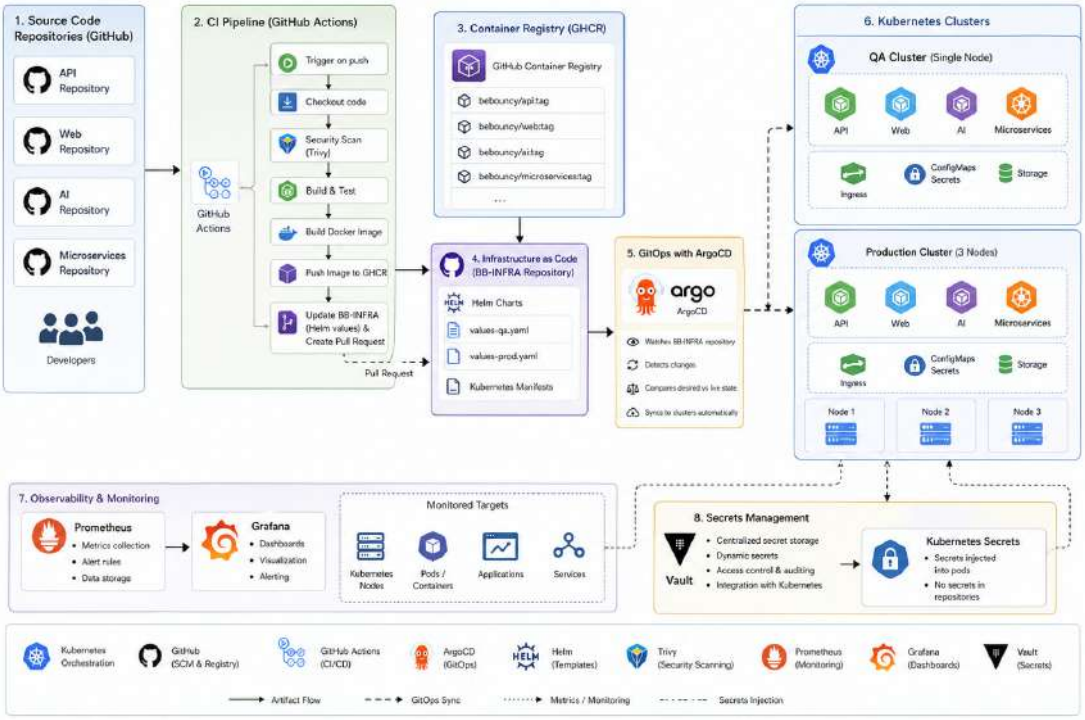


Figure 1.11: Proposed DevSecOps Architecture

Table 1.4 summarizes the before/after comparison at a glance, highlighting the key operational, security, and scalability improvements introduced by the proposed architecture.

Challenge	**Before (PM2 on VPS)**	**After (Docker + Kubernetes + GitOps)**
Deployment	Auto and Manual PM2 Deployments, error-prone	Automated via Git push; pipeline handles everything
Environment Consistency	QA and production diverge over time	Same container image runs identically everywhere
Rollback	Manually redeploy previous code (minutes to hours)	Run Rollback pipeline to revert in seconds
Secrets Management	`.env` files scattered across repos	Vault + Kubernetes Secrets, centralized and encrypted
Scaling	Manual PM2 adjustment	Horizontal Pod Autoscaler based on CPU/memory
Failure Recovery	PM2 restarts process; machine crash requires manual intervention	Kubernetes reschedules pods onto healthy nodes automatically
Security	No automated scanning	Trivy, SonarQube Cloud, Checkov, and OWASP ZAP scans on every build; secrets never in images.
Observability	PM2 logs only, no alerting	Prometheus + Grafana dashboards with proactive alerts

Table 1.4: Impact of the Proposed Solution

1.6 Requirements Identification

1.6.1 System Actors

In the context of a DevSecOps platform, the concept of an "actor" extends beyond end-users to encompass any human role or automated agent that interacts with the infrastructure. Three primary

actors have been identified:

Figure 1.12: System Actors

1.6.2 Functional Requirements

Functional requirements define the capabilities that the DevSecOps system must provide. Each requirement is derived directly from the limitations identified in the existing infrastructure analysis presented in Section 4.

ID	Category	Requirement
1	CI/CD Automation	A build and test pipeline must trigger automatically on every code push, requiring no manual developer intervention.
2	Security Scanning	The pipeline must scan source code and Docker images for vulnerabilities using security gates prior to any artifact being pushed to the registry.
3	Secret Detection	Commits containing hardcoded secrets, API keys, or credentials must be automatically detected and blocked before merging into the main branch.

(Continued on next page)

ID	Category	Requirement
4	Container Build	The system must build reproducible, multi-stage Docker images for all BeBouncy services (API, Web, AI, Microservices), guaranteed to behave identically across QA and production.
5	Image Registry	Validated images must be automatically pushed to GHCR and tagged with the triggering commit SHA for full traceability.
6	GitOps Deployment	All deployments must be driven by Git commits to `BB-INFRA`. ArgoCD must continuously reconcile cluster state to the declared configuration, replacing all manual SSH-based deployments.
7	Infrastructure as Code	All Kubernetes resources must be defined as Helm chart templates parameterized by environment-specific values files, ensuring full cluster reproducibility from Git alone.
8	Rollback	Any deployment must be reversible by running a pipeline to revert relevant commit in `BB-INFRA`, with ArgoCD automatically propagating the change to the cluster.
9	Horizontal Scaling	The cluster must support HPA rules that automatically adjust pod replicas based on CPU and memory utilization thresholds, without manual intervention.
10	Secret Management	All secrets and credentials must be stored in HashiCorp Vault and injected at runtime via Kubernetes Secrets; plaintext secrets must never appear in repository files.

(Continued on next page)

ID	Category	Requirement
11	Metrics Collection	Prometheus must collect infrastructure and application-level metrics from all nodes and pods, exposed through Grafana dashboards.
12	Alerting	The system must trigger automated alerts when key thresholds are exceeded (e.g., CPU > 80%, pod crash-looping, disk > 85%) and notify the operations team via a configured channel.

Table 1.5: Functional Requirements of the DevSecOps System

1.6.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints that the system must satisfy, independent of any specific feature. In a DevSecOps context, these constraints directly govern how reliable, secure, and sustainable the infrastructure will be.

- **Performance:** CI/CD pipeline execution time must remain within acceptable bounds so as not to impede development velocity. Container image builds must be optimized through multi-stage Dockerfiles and layer caching to minimize build duration. Production deployments must complete with zero application downtime, achieved through Kubernetes rolling update strategies.
- **Security:** Security controls and security gates must be integrated throughout every stage of the pipeline, following a shift-left security approach. Container images containing critical or high-severity CVEs must not be promoted to production. Access to the Kubernetes API server and to Vault must be enforced through Role-Based Access Control (RBAC) policies aligned with the principle of least privilege.
- **Reliability and High Availability:** The production cluster must be configured with a minimum of three nodes to provide fault tolerance. Pod disruption budgets must ensure that rolling updates never reduce service availability below a defined minimum replica count. Kubernetes self-healing mechanisms must automatically reschedule failed pods onto healthy nodes.
- **Scalability:** The infrastructure must handle increased traffic without requiring manual reconfiguration. The Horizontal Pod Autoscaler must respond to load variations within a configurable time window, scaling both up and down to optimize resource consumption.

- **Maintainability and Reproducibility:** The entire infrastructure state must be expressible as versioned code in Git. Any engineer must be able to reproduce the full cluster environment from a clean machine using only the contents of the `BB-INFRA` repository. Configuration drift must be structurally impossible, as ArgoCD continuously enforces the declared state.

- **Observability:** The system must provide sufficient telemetry to allow the operations team to understand the internal state of every service without requiring direct server access. This implies the availability of real-time metrics dashboards, structured log aggregation, and proactive alerting, replacing the current practice of consulting raw PM2 logs manually.

1.7 Proposed System Design

1.7.1 Pipeline Architecture

The CI/CD pipeline constitutes the central nervous system of the DevSecOps solution. Figure 1.13 details the sequential stages executed by GitHub Actions from a code push to a live deployment, including the security gates integrated at each transition point.

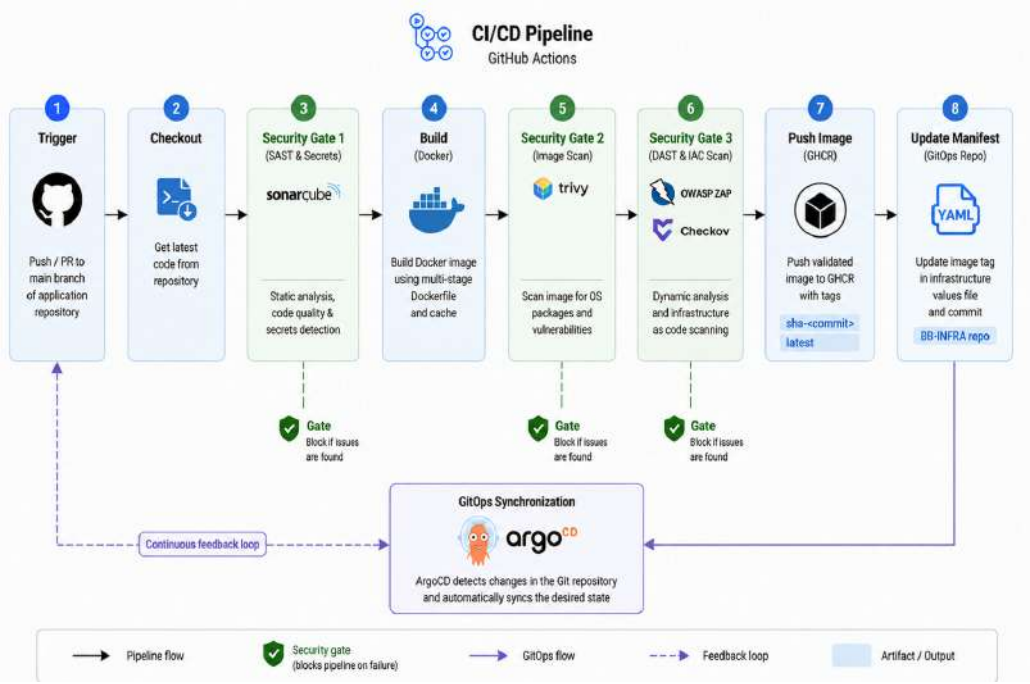


Figure 1.13: CI/CD Pipeline Flow Diagram

The pipeline stages are:

1. **Trigger:** A push or pull request to the `main` branch of the application repository initiates the GitHub Actions workflow.

2. **Checkout:** The workflow checks out the latest source code

3. **Source Scan:** Trivy performs a filesystem scan to detect vulnerable dependencies and exposed secrets, while SonarQube conducts static code analysis for code quality and security issues.

4. **Build:** The Docker image is constructed using the multi-stage Dockerfile. Build cache is leveraged to minimize execution time.

5. **Image Scan:** Trivy scans the freshly built image for OS-level and library-level CVEs. The pipeline fails and halts if any critical vulnerability is detected.

6. **IaC Security Validation:** Checkov scans IaC configurations for security misconfigurations and policy violations. The pipeline halts if critical issues are identified.

7. **Push:** The validated image is pushed to GHCR, tagged with the commit SHA and a `latest` floating tag.

8. **Update Manifest:** An automated commit updates the image tag in the corresponding `BB-INFRA` values file, triggering ArgoCD synchronization for the QA environment.

9. **Post-Deployment DAST:** Once ArgoCD has deployed the image to the QA cluster, OWASP ZAP executes dynamic security testing against the live QA endpoints to identify runtime web vulnerabilities. Results are reported to the team; critical findings block production promotion.

1.7.2 Logical Architecture

The logical architecture organizes the system into distinct functional layers that interact through well-defined interfaces:

- **Source Control Layer (GitHub):** All application code and infrastructure declarations reside in Git repositories. This layer is the single source of truth for both the application and its deployment configuration.

- **Automation Layer (GitHub Actions):** Pipelines execute on GitHub-hosted runners and orchestrate the build, scan, test, and publish sequence. This layer has no direct access to the production cluster; it communicates only with GHCR and the `BB-INFRA` repository.

- **Registry Layer (GHCR):** The GitHub Container Registry stores all validated container images, versioned by commit SHA. Only images that have passed all security gates are present in this registry.

- **GitOps Layer (ArgoCD):** ArgoCD continuously polls `BB-INFRA` and reconciles the live cluster state to match. This layer provides the sole authorized path to the Kubernetes clusters.

- **Orchestration Layer (Kubernetes):** Application workloads run as pods managed by Kubernetes, with full benefit of rolling updates, self-healing, and horizontal autoscaling.
- **Security Layer:** Security is enforced across every phase of the pipeline rather than at a single point. Static analysis and code quality gates are applied at the source level, container images are scanned for vulnerabilities and assessed against a generated software bill of materials, infrastructure-as-code definitions are validated for misconfigurations, and dynamic analysis targets the running application. Secret management and cluster access controls complete the chain, ensuring that no unvetted artifact or unauthorized principal reaches the production environment.
- **Observability Layer:** Metrics are scraped from all cluster components and exposed through Grafana. Alerting rules forward threshold violations to the operations team.

1.7.3 Physical Architecture

The physical infrastructure underpinning the DevSecOps solution is deployed across OVHcloud virtual machines provisioned with Ubuntu 24.04. The production environment consists of a three-node Kubernetes cluster (one control plane node and two worker nodes), providing fault tolerance and the ability to drain and maintain individual nodes without service interruption. The QA environment runs as a single-node Kubernetes cluster on a dedicated machine, mirroring the production configuration at reduced scale.

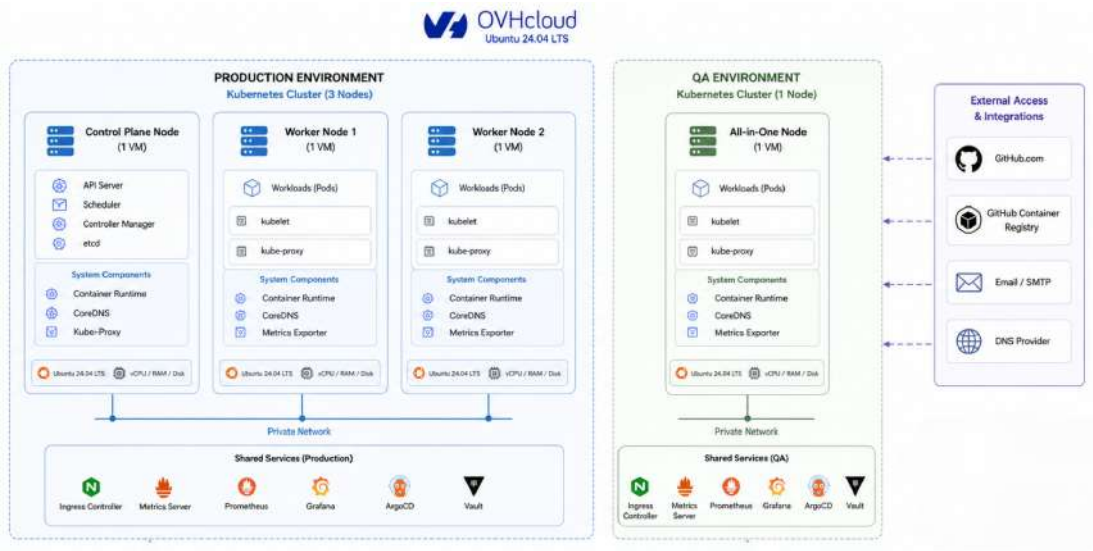


Figure 1.14: Physical Architecture of the DevSecOps Infrastructure

Having established the system architecture, the next step is to define the methodology adopted for the implementation of the project.

1.8 Adopted Methodology

1.8.1 The Importance of Choosing a Methodology

To ensure the success of this project, it is essential to select an appropriate development and project management methodology that structures the work effectively and adapts to the evolving needs of the organization. This choice is critical for aligning the execution process with the specific technical requirements of a DevSecOps transformation and the strategic objectives of a startup in rapid growth.

1.8.2 Agile Approach

The progression of this project has been guided primarily by Agile principles, which offer increased flexibility, foster close collaboration with stakeholders, and allow for in-course adjustments as new information emerges. The nature of a DevSecOps project where infrastructure decisions have cascading effects across teams and where tooling choices may need to be revised as deployment constraints become clearer which makes agility not just a preference but a necessity.

Rather than committing to a fixed, waterfall-style plan that would be difficult to adjust when encountering unexpected infrastructure constraints or API limitations, the Agile approach allowed the team to deliver working increments at each sprint while continuously incorporating feedback from developers and operations staff.

1.8.3 Comparison of Agile Methods: Scrum, Kanban, and Scrumban

Three Agile methods were evaluated before selecting the one best suited to the project's context:

Aspect	**Scrum**	**Kanban**	**Scrumban**
Methodology	Iterative development framework with defined sprints	Lean method for continuous flow and improvement	Hybrid combining Scrum structure with Kanban flexibility
Iteration Duration	Fixed sprints of 2–4 weeks	No fixed cycles; continuous flow	Flexible, adapted to team needs
Roles	Scrum Master, Product Owner, Development Team	No defined roles	Similar to Scrum but more flexible
Meetings	Daily stand-ups, planning, review, retrospective	No fixed meetings; periodic reviews	Similar to Scrum but fewer ceremonies
Flow Tracking	Scrum board for sprint planning	Kanban board for workflow visualization	Visual boards as in Kanban
Flexibility	Medium: changes go through the backlog	Very high: easy to adapt at any time	High: changes possible even mid-sprint
Primary Goal	Deliver defined scope within fixed iterations	Continuous delivery and bottleneck reduction	Balance between iteration structure and continuous flow

Table 1.6: Comparison of Agile Methods: Scrum, Kanban, and Scrumban

1.8.4 Chosen Method: Scrumban

After evaluating the three methods, Scrumban was selected as the project management methodology for this internship project. Scrumban is a hybrid approach that combines the iterative structure of Scrum with the visual flexibility of Kanban.

The rationale for this choice is grounded in the specific nature of the project:

Factor	Justification
Nature of the project	The mission involves designing and implementing a DevSecOps infrastructure across multiple layers (CI/CD, containerization, orchestration, security, monitoring). Tasks are interdependent and may unblock or reorder one another as the project progresses.
Individual role with team coordination	As the sole DevSecOps engineer intern, autonomous decision-making is required. However, coordination with the development team is necessary to validate pipeline integration points and deployment conventions.
Evolving technical requirements	Infrastructure tooling decisions (e.g., choice of ingress controller, secret management approach, monitoring stack) may change as constraints are discovered during implementation. Scrumban allows these adjustments without halting the sprint.
Urgency handling	If a critical environment issue arises mid-sprint such as a disk saturation event on the QA server or an unexpected Kubernetes API incompatibility, it can be addressed immediately without disrupting other ongoing tasks, thanks to Kanban's pull system.

Table 1.7: Justification for Choosing Scrumban

1.8.5 Scrumban Rituals

The following ceremonies were adopted throughout the project to maintain structure while preserving flexibility:

Figure 1.15: Scrumban Rituals Adopted for This Project

1.9 Sprint Planning

The project is organized into iterative development cycles following the Scrumban methodology adopted in Section 8. Each cycle targets a cohesive infrastructure layer spanning containerization, CI/CD automation, container orchestration, GitOps deployment, security integration, and observability to ensure that every deliverable is functional and testable before the next layer is built upon it. This incremental structure mirrors the dependency chain inherent to a DevSecOps pipeline, where later phases rely on the stability of earlier ones, making sequencing a core design decision. Figure 1.16 presents the project planning board used to track tasks, priorities, and development progress throughout the implementation phase.

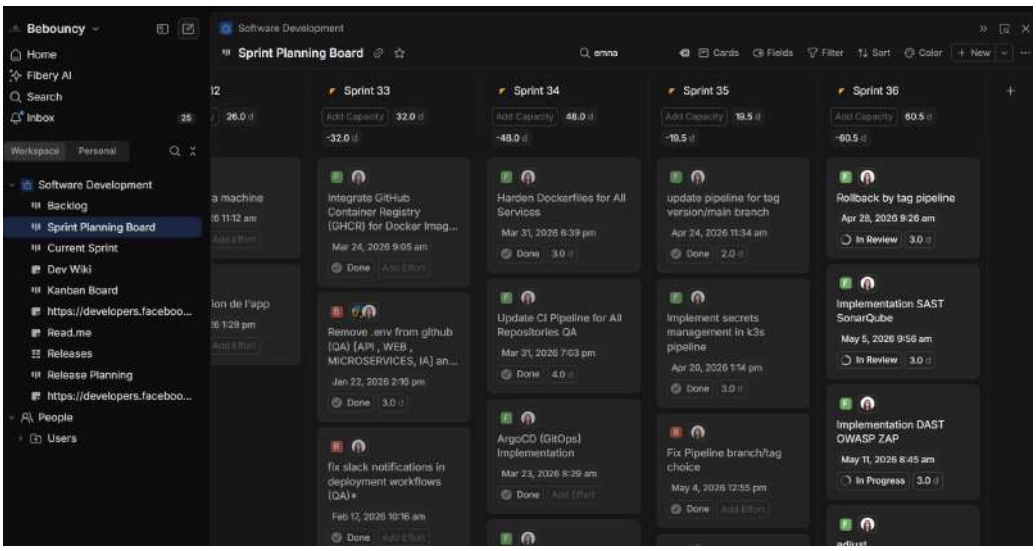


Figure 1.16: Overview of Fibery's Sprint Planning Board

1.9.1 Project Timeline

Figure 1.17 presents the project timeline and the distribution of implementation tasks throughout the internship period. The initial phases establish the foundational layers, with containerization and CI/CD automation serving as prerequisites for subsequent work, while later phases progressively introduce orchestration, GitOps delivery, security controls, and full-stack observability on top of a validated base.

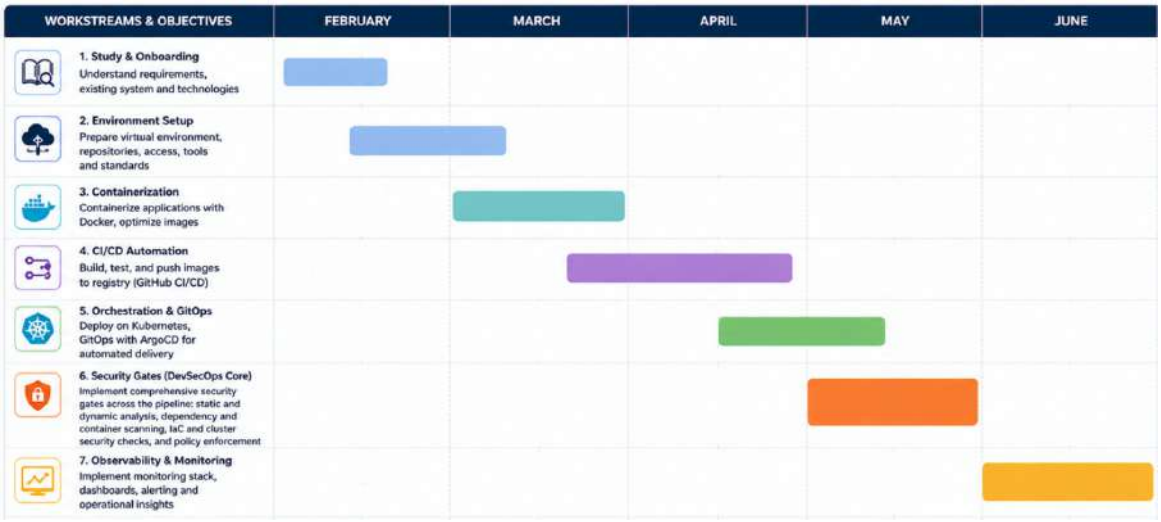


Figure 1.17: Gantt chart of the project execution timeline

1.10 Development Environment and Tools

The technology stack selected for this project is the result of deliberate choices driven by the specific requirements of a startup-scale DevSecOps environment, the existing skill sets within the team, and the maturity and community support of each tool.

Tool	Description
 Figure 1.18: Docker Logo	Containerization platform enabling portable and reproducible environments.

(Continued on next page)

Tool	Description
 Figure 1.19: Kubernetes Logo	Orchestrates containers and manages deployment, scaling, and availability.
 Figure 1.20: GitHub Actions Logo	Automates CI/CD pipelines directly within GitHub repositories.
 Figure 1.21: ArgoCD Logo	Implements GitOps-based continuous deployment with full traceability.
 Figure 1.22: Helm Logo	Manages Kubernetes applications using reusable and configurable charts.
 Figure 1.23: Trivy Logo	Scans containers and code for vulnerabilities and misconfigurations.
 Figure 1.24: HashiCorp Vault Logo	Securely stores and manages secrets with access control and auditing.

(Continued on next page)

Tool	Description
 Figure 1.25: SonarQube Logo	Performs static code analysis to detect bugs, code smells, and security vulnerabilities, enforcing code quality standards throughout the development lifecycle.
 Figure 1.26: Prometheus Logo	Collects, stores, and queries time-series metrics from systems, enabling performance monitoring and alerting.
 Figure 1.27: Grafana Logo	Visualizes metrics and provides dashboards with alerting capabilities.
 Figure 1.28: Slack Logo	Team collaboration platform that centralizes communication through channels, direct messaging, file sharing, and integrations with other tools.
 Figure 1.29: Fibery Logo	All-in-one collaborative platform that combines documentation and project management.
 Figure 1.30: Visual Studio Code Logo	Lightweight and extensible source-code editor used for application development, configuration management, and DevOps workflow integration.

Table 1.8: Technology Stack Overview

Conclusion

This chapter established the context and planning framework of the internship project. BeBouncy was introduced as an AI-driven social media management startup, followed by an analysis of its existing infrastructure and the identification of key limitations related to automation, security, observability, and deployment consistency. Based on these findings, a DevSecOps-oriented solution was proposed.

The chapter also presented the functional and non-functional requirements, the logical and physical architectures of the platform, the adopted Scrumban methodology, and the selected technology stack with the justification behind each choice.

The following chapter will present the state of the art related to containerization, DevOps, DevSecOps, CI/CD, and GitOps approaches, followed by the proposed architecture and the technical comparison and justification of the selected technologies.

Chapter 2

State of the Art, Architecture and Technical Comparison

Contents

Introduction

This chapter presents the theoretical foundation and architectural decisions that underpin the DevSecOps infrastructure designed for BeBouncy. It begins with a state-of-the-art review of the key technologies involved: virtualization, containerization, DevOps, DevSecOps, CI/CD, and GitOps which establishes the conceptual groundwork for the choices made throughout the project. It then describes the overall architecture of the proposed solution, detailing the CI/CD pipeline flow and the Kubernetes deployment topology. Finally, it provides a systematic comparative analysis of the tools evaluated in each category, justifying the selection of the specific technologies that constitute the BeBouncy DevSecOps stack.

2.1 State of the Art

Before designing the infrastructure, it is important to conduct a state-of-the-art review. This step involves analyzing existing works, methods, and technologies in the field in order to identify best practices and effective solutions. It enables a better understanding of the technical context, a comparison of available approaches, and lays the groundwork for designing an architecture that is both appropriate and performant for the project.

2.1.1 Virtualization and Containerization

Virtualization and containerization are two modern approaches that allow multiple applications to be deployed on a single physical server. They differ in their operating mode, resource consumption, and execution speed.

2.1.1.1 Virtualization

Virtualization relies on the use of a **hypervisor** to create multiple Virtual Machines (VMs) on a single physical server. Each VM has its own operating system, its own allocated resources, and functions as an independent computer.

This approach optimizes the use of physical infrastructure but consumes more resources (memory, CPU, storage). Furthermore, the startup time of a VM is relatively slow because each VM must load a complete operating system.

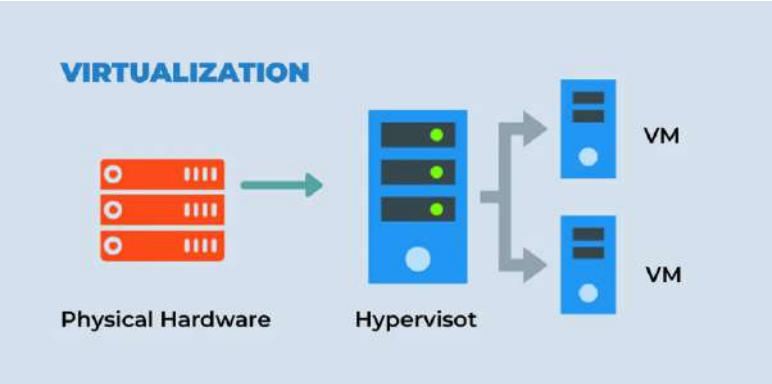


Figure 2.1: Principle of Virtualization

2.1.1.2 Containerization

Containerization allows applications to run inside isolated **containers** that share the same host operating system kernel. Each container packages only the application and its dependencies (libraries, configurations), making them very lightweight and fast to start.

This approach consumes fewer resources than virtualization and facilitates the rapid, automated deployment of applications, particularly in a DevOps context.

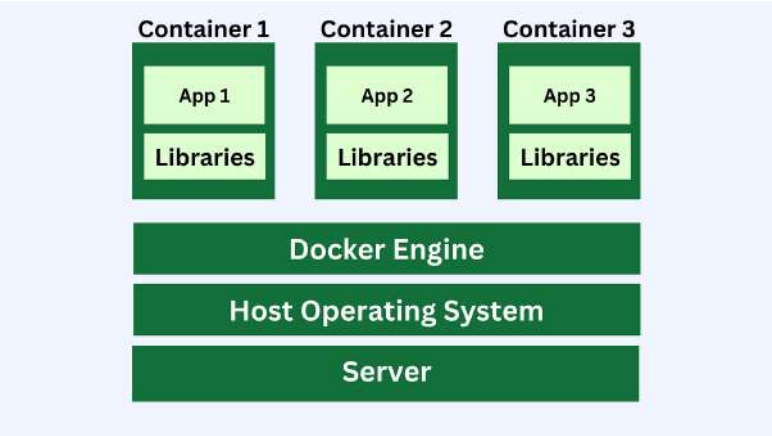


Figure 2.2: Principle of Containerization

2.1.2 Comparison Between Virtualization and Containerization

Table 2.1 highlights the main differences between virtualization and containerization, emphasizing their respective characteristics and advantages.

Criterion	Virtualization	Containerization
Isolation	Each virtual machine has its own fully separate operating system	Containers share the host kernel, with software-level isolation
Resource Usage	High consumption of RAM, CPU, and storage	Lighter; optimized consumption through resource sharing
Startup Time	Relatively slow (full OS boot required)	Very fast (a few seconds)
Portability	Less flexible; often dependent on the VM environment	Highly portable; ideal for cloud and multi-platform environments
Use Cases	Environments requiring strong isolation or support for diverse operating systems	Rapid deployment, microservices, DevOps, CI/CD

Table 2.1: Comparison Between Virtualization and Containerization

The comparison above highlights the efficiency and flexibility of containerization for modern cloud-native environments. Building on these foundations, the following section introduces the DevOps, DevSecOps, CI/CD, and GitOps approaches.

2.2 DevOps, DevSecOps, CI/CD, and GitOps Approaches

2.2.1 DevOps Approach

DevOps is an approach aimed at bringing together development teams (*Dev*) and operations teams (*Ops*). The objective is to work together more efficiently in order to deliver software faster and with higher quality.

DevOps rests on three essential principles:

- **Collaboration** between teams,
- **Automation** of repetitive tasks,
- **Continuous integration** of code changes.

By automating processes such as testing, deployments, and monitoring, DevOps enables teams to detect and fix errors faster, improve code quality, and reduce the time-to-market for new features.

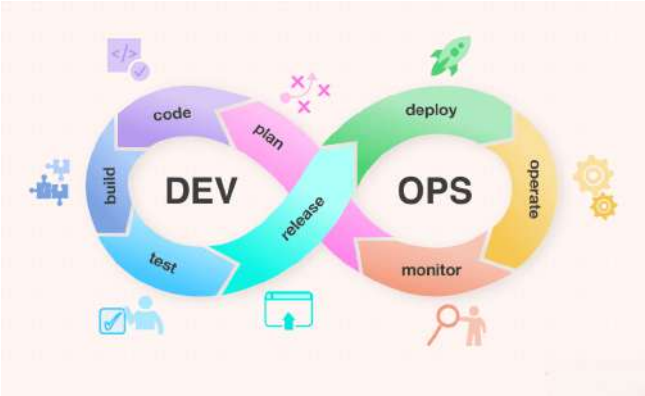


Figure 2.3: The DevOps Cycle

2.2.2 Advantages of DevOps

Adopting DevOps brings numerous benefits:

- Better communication and shared responsibility between teams,
- Reduction of errors through automation,
- Faster and more frequent deployments,
- Continuous improvement driven by user feedback and real-time monitoring.

2.2.3 DevSecOps Approach

DevSecOps is an extension of DevOps that integrates security from the very first stages of development. Unlike the traditional approach where security is added at the end, DevSecOps involves security teams from the outset. The objective is to embed security practices and tools throughout the entire development lifecycle, a principle commonly referred to as **Shift Left Security**.

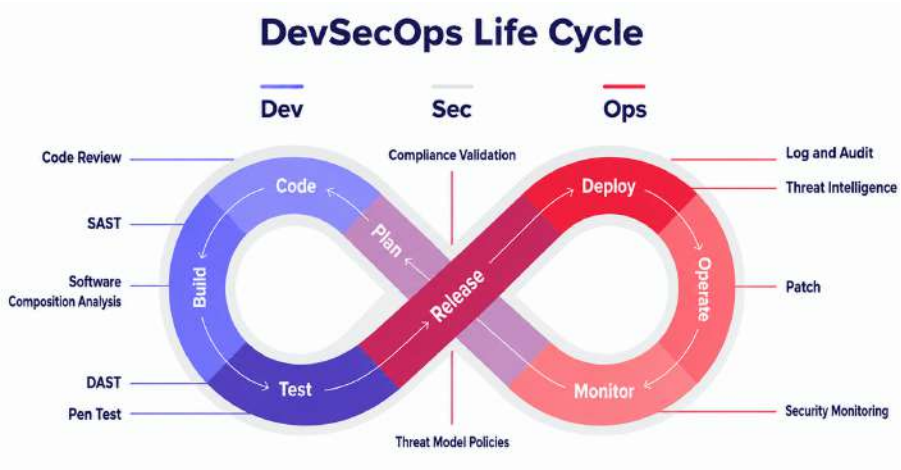


Figure 2.4: DevSecOps: Security Integrated Throughout the Development Lifecycle

2.2.3.1 DevOps vs. DevSecOps

Table 2.2 highlights the main differences between DevOps and DevSecOps, emphasizing their respective objectives and practices.

Aspect	**DevOps**	**DevSecOps**
Primary Objective	Automate and accelerate application development and deployment	Add security at every stage of the software lifecycle
Teams Involved	Developers + Operations	Developers + Operations + Security Teams
Security Approach	Security treated at the end of the cycle (late addition)	Security integrated from the beginning of the project (Shift Left)
Responsibility	Security managed by a dedicated team	Security shared across all stakeholders
Delivery	Fast and continuous	Fast, continuous, and secure
Advantages	Collaboration + Automation + Speed	Collaboration + Automation + Speed + Proactive Security + Reduced Vulnerabilities

Table 2.2: Comparison Between DevOps and DevSecOps

The comparison in Table 2.2 illustrates how DevSecOps extends the traditional DevOps approach by integrating security throughout the entire development lifecycle. To support this continuous and automated workflow, organizations rely on CI/CD practices.

2.2.4 CI/CD Approach

The **CI/CD** (Continuous Integration and Continuous Deployment) approach is a key element of the DevOps methodology. It enables the automation of testing and deployments in order to deliver software rapidly, frequently, and with greater reliability.

1. **Continuous Integration (CI):** Continuous integration consists of regularly merging code

changes made by different developers into a main branch of the project. Each change is automatically validated through automated tests, enabling teams to:

- **Detect errors quickly:** by testing each change, bugs are identified and fixed earlier;
- **Improve code quality:** tests ensure that each update respects defined quality standards;
- **Reduce conflicts:** by merging code frequently, conflicts between developers' work are avoided.

2. **Continuous Deployment (CD):** Continuous deployment goes further: each change that has been validated and tested automatically is then pushed to production without manual intervention. Key aspects include:

   - **Full automation:** from code commit to production deployment, everything is automated;
   - **Rapid delivery:** new features or bug fixes can be deployed multiple times per day;
   - **Active monitoring:** tools continuously watch production applications to detect errors and intervene quickly when needed.

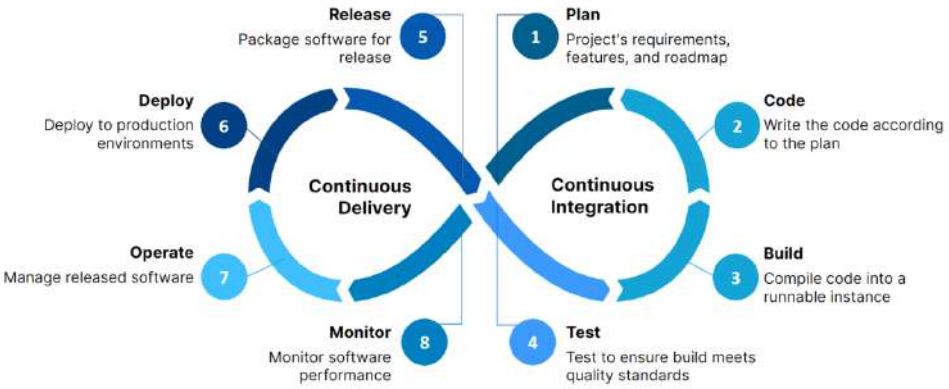


Figure 2.5: CI/CD Pipeline

2.2.5 GitOps

GitOps is an approach that uses Git as the single source of truth for managing deployments and application configuration. Every change to the infrastructure or application is made via a Git commit, and a GitOps tool (such as ArgoCD) automatically synchronizes the real state of the Kubernetes cluster with what is defined in Git. This methodology centralizes configuration management, automates updates, and guarantees that the cluster state always matches what is declared in the repository.

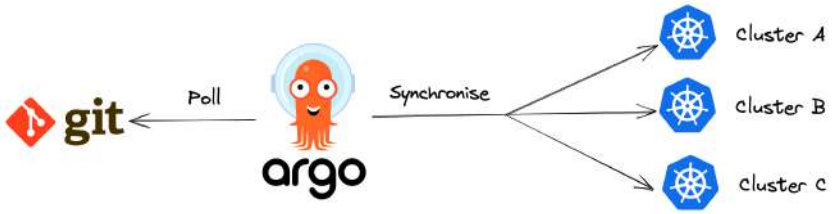


Figure 2.6: GitOps Workflow with ArgoCD

2.2.5.1 Difference Between GitOps and DevOps

Table 2.3 presents a concise comparison between the DevOps and GitOps approaches, highlighting the main differences in terms of objectives, sources of truth, automation, technologies, and focus.

Criterion	**DevOps**	**GitOps**
Objective	Team collaboration	Infrastructure automation via Git
Source of Truth	Application and infrastructure configuration	Git repository
Automation	Partial	Strong; automated deployment
Technologies	All environments	Containers, Kubernetes
Focus	Process and culture	Infrastructure as Code, deployments
Advantage	Communication	Infrastructure conformity to code

Table 2.3: Compact Comparison Between GitOps and DevOps

The comparison in Table 2.3 highlights how GitOps complements DevOps by strengthening automation and infrastructure consistency through Git-based workflows. Based on these concepts, the following section presents the architecture and design of the proposed solution.

2.3 Architecture and Design

This section presents the system architectures and the design choices adopted for the BeBouncy DevSecOps infrastructure. It describes the various components, their interactions, and the way in which they are integrated to ensure performance, scalability, and maintainability of the solution.

2.3.1 Automated Deployment Architecture

The automated deployment architecture illustrates how the end user accesses the application through an Ingress Controller (or NodePort service in the k3s setup). The CI/CD pipeline, built on GitHub Actions, generates Docker images that are then stored in the GitHub Container Registry (GHCR). The automated deployment of these images onto the Kubernetes cluster is managed by ArgoCD.

The production cluster is composed of a control-plane node running the core Kubernetes components (API Server, Controller Manager, Scheduler, etcd), and two worker nodes hosting the application pods. A dedicated QA cluster mirrors this setup at reduced scale (single node). All virtual machines are provisioned on OVHcloud running Ubuntu 24.04.

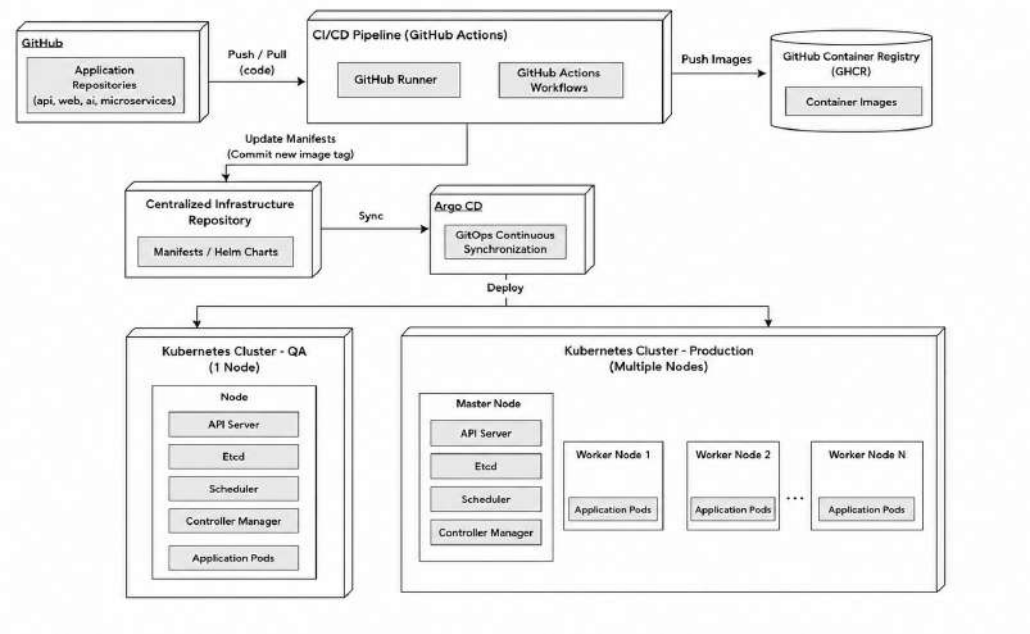


Figure 2.7: Automated Deployment Architecture

2.3.2 Global Solution Architecture

The global architecture integrates several key components into a coherent end-to-end solution. It includes, first, a CI/CD pipeline based on GitHub Actions that handles the build, testing, security scanning, and deployment steps while storing validated images in GHCR. ArgoCD then takes charge of the continuous synchronization of changes to the Kubernetes cluster. GitHub Actions also automatically updates the `BB-INFRA` manifest repository with the new image tag, triggering ArgoCD reconciliation.

Finally, a monitoring stack based on Prometheus and Grafana provides real-time supervision of the infrastructure and applications, with alerting rules forwarding threshold violations to the operations team via Slack.

Figure 2.8 presents the global architecture of the solution, from code push through to deployment and production monitoring.

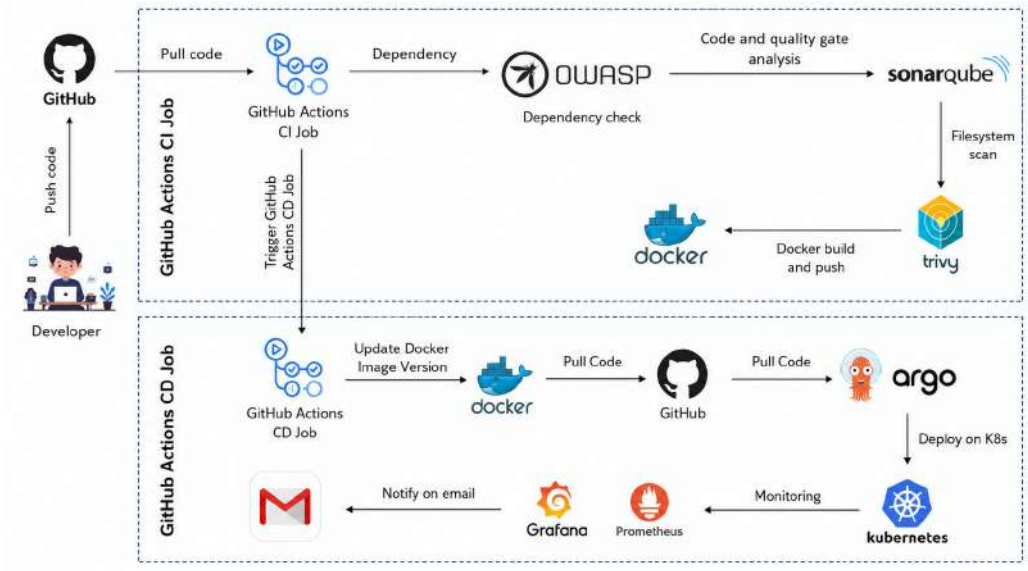


Figure 2.8: Global Architecture of the Solution

2.3.3 CI/CD Pipeline

The CI/CD pipeline automates the development, validation, security enforcement, and deployment of the BeBouncy application. The pipeline stages are as follows:

- **Commit:** A push to the `main` branch of an application repository triggers the GitHub Actions workflow.
- **Build and Tests:** The application is built and tested. If any test fails, the pipeline halts immediately, preventing broken code from advancing.
- **Code Quality Analysis (SonarQube):** Static analysis is performed to detect bugs, code smells, and security vulnerabilities in the source code.
- **Docker Image Build:** Docker images are generated using multi-stage Dockerfiles and pushed to the GitHub Container Registry (GHCR).
- **Security Analysis:** Images are scanned for known vulnerabilities. If critical or high-severity CVEs are detected, the pipeline halts and no artifact reaches the registry.
- **Manifest Update:** An automated commit updates the image tag in the `BB-INFRA` infrastructure repository, triggering ArgoCD synchronization for the QA environment.
- **Deployment:** ArgoCD deploys the validated images onto the Kubernetes cluster. Promotion to production requires a pull-request review and explicit team approval before ArgoCD propagates the change.

This process ensures quality, security, and continuous deployment, with automatic pipeline halting in the event of critical risks.

2.3.4 Deployment Architecture

The QA deployment architecture relies on Kubernetes for cluster orchestration and resource management. Application deployments are managed by ArgoCD within the dedicated `qa` namespace.

QA Node (Single Worker Node): hosts both the Kubernetes workloads and the application services deployed for the QA environment. The node runs the following pods:

- API and AI service pods,
- Web front-end (Next.js) pods,
- Microservices pods
- Monitoring and observability pods.

The architecture of the QA environment provides a controlled platform for validating deployment workflows, infrastructure configurations, and security mechanisms before their adoption in production environments. Based on these operational and security requirements, it is essential to justify the technological choices retained for the implementation of the DevSecOps and GitOps ecosystem.

2.4 Analysis and Justification of Technological Choices

In the context of implementing DevSecOps and GitOps practices, the selection of appropriate tools and platforms represents a strategic aspect of the infrastructure and deployment lifecycle. These technological choices directly influence the efficiency, security, scalability, and reliability of software delivery processes, while also impacting maintainability and operational consistency across environments. As modern cloud-native architectures rely on automation and continuous integration and deployment mechanisms, it becomes essential to adopt solutions that facilitate orchestration, security enforcement, infrastructure management, and deployment traceability.

This section provides a comparative and analytical study of the principal technologies and platforms available for implementing the proposed DevSecOps and GitOps ecosystem. The objective is to evaluate their features, advantages, and limitations in order to identify the solutions that best satisfy the technical, operational, and security requirements of the project.

2.4.1 Version Control Tools

Version control is essential for ensuring collaboration between developers and tracking changes throughout the software development lifecycle. Several tools were studied for their efficiency, their features, and their ability to integrate into the project's workflow.

Criterion	**Git (GitHub/GitLab)**	**SVN**	**Mercurial**
Type	Distributed	Centralized	Distributed
Complexity	Medium to high	Low	Low
Collaboration	Very effective	Limited	Medium
CI/CD Support	Yes, integrated	Not native	Limited
Popularity	Very high	Medium	Low

Table 2.4: Comparison of the Most Widely Used Version Control Tools

For this project, **GitHub** was chosen as the version control solution. This platform combines the power of Git with integrated collaboration tools, change tracking, and branch protection rules. It also enables seamless integration with GitHub Actions for CI/CD pipelines, meeting the technical and organizational needs of the project.

2.4.2 Containerization Tools

In a DevSecOps environment, the choice of containerization tool is crucial for ensuring portability, security, and application performance. Each solution has specific characteristics that may guide the selection based on project requirements.

Criterion	Docker	CRI-O	containerd
Role	Image, network, and volume management	Lightweight runtime for Kubernetes	Modular runtime (Docker/K8s)
License	Apache 2.0	Apache 2.0	Apache 2.0
Platform	Linux, Windows, macOS	Linux	Primarily Linux
Security	Basic by default	Rootless, integrated with K8s	Rootless, enhanced protection
CLI	Full CLI (build/push/pull)	Via Buildah/Skopeo	CLI/API, external build
Target	Developers, DevOps	Kubernetes administrators	Advanced cloud/DevOps
K8s Integration	Docker Shim (deprecated)	Native CRI	Native CRI

Table 2.5: Comparison of Containerization Tools

Docker was chosen for its simplicity, rich feature set, and wide adoption, facilitating image, network, and volume management for all four BeBouncy services (API, Web, AI, Microservices). For the Kubernetes cluster runtime, **containerd** is used natively by k3s, ensuring a lightweight and CRI-compliant container runtime with enhanced isolation.

2.4.3 Container Orchestration Tools

To manage container orchestration, several tools were analyzed and compared based on their scalability, reliability, ease of management, and compatibility with DevSecOps and GitOps practices. The evaluation also considered their ability to support automated deployments, resource management, and cloud-native application environments, with the objective of selecting the solution that best fits the project requirements.

Tool	Advantages	Disadvantages
Kubernetes	Highly scalable, large community, rich ecosystem, native CI/CD and DevOps support	Configuration complexity; steep learning curve
Docker Swarm	Easy to install and use; integrated with Docker	Less flexible than Kubernetes; less suited for large clusters
OpenShift	Kubernetes-based, enhanced security, integrated DevOps tools	Complex; requires more resources; commercial license for some features
Nomad	Lightweight, simple to deploy, multi-cloud	Fewer native features than Kubernetes; smaller community

Table 2.6: Comparison of Main Container Orchestration Tools

Kubernetes was selected as the orchestration solution for its robustness, scalability, and ability to support modern architectures. Despite its deployment complexity, it remains the widely adopted industry standard. k3s was specifically chosen because it is optimized for resource-constrained environments such as the OVHcloud VMs used by BeBouncy, while retaining full Kubernetes API compatibility.

2.4.4 Code Analysis Tools

To guarantee code quality and security throughout the project, it is essential to use appropriate static analysis tools. These allow bugs and vulnerabilities to be detected from the earliest phases of development, while verifying compliance with quality standards and security best practices.

Tool	Advantages	Limitations
SonarQube	Supports 25+ languages, intuitive web interface, CI/CD integration (GitHub Actions, Jenkins, GitLab)	Free version is limited; heavy configuration on large projects
Snyk Code	Rapid vulnerability detection, GitHub/GitLab integration, fix suggestions	Reduced language support; full version is paid
CodeQL	Powerful custom queries, GitHub Actions integration, free for public repos	High learning curve; oriented toward GitHub

Table 2.7: Advantages and Limitations of Code Analysis Tools

Among the tools compared, **SonarQube** was retained for its broad language coverage (the BeBouncy codebase uses Node.js/TypeScript and Python), its ease of integration into GitHub Actions pipelines, and its intuitive interface. It enables effective code quality tracking and vulnerability detection from the earliest stages of development.

2.4.5 Docker Image Security Analysis Tools

Security analysis of Docker images is essential for detecting vulnerabilities and ensuring that deployed containers are safe. These tools scan images, identify potential flaws, and verify that security best practices are respected prior to deployment.

Criterion	Trivy	Snyk Container	Clair
Analysis Type	Image scan, dependencies, filesystem	Image + dependency scan, alerts and fixes	CVE-based vulnerability analysis
Ease of Use	Simple CLI, easy integration	CLI + web, advanced dashboards	Technical, manual configuration
Reports	CVE details + fixes	Full reports, suggested patches	Raw CVE-based reports
CI/CD Integration	Excellent (GitHub Actions, GitLab, Jenkins)	Very good via plugins	Possible but poorly documented
License	100% Open Source	Freemium / commercial	Open Source (CoreOS)

Table 2.8: Comparison of Docker Image Security Analysis Tools

Trivy was chosen for Docker image security analysis due to its simplicity, speed, and seamless integration into GitHub Actions pipelines. Being 100% open source, it provides detailed vulnerability reports and is perfectly suited to a DevSecOps approach.

2.4.6 Continuous Integration (CI) Tools

Continuous Integration (CI) plays a fundamental role in modern DevSecOps practices by automating code validation, testing, and build processes throughout the software development lifecycle. Selecting an appropriate CI solution is essential to ensure fast feedback cycles, deployment reliability, and efficient integration with containerized and cloud-native environments. In this context, several CI tools were examined and compared according to criteria such as ease of configuration, automation capabilities, Docker integration, maintenance requirements, and compatibility with the existing development ecosystem.

Criterion	GitHub Actions	Jenkins	GitLab CI/CD
Git Integration	Native GitHub	Plugins required	Native GitLab
Configuration	Simple (`.github/workflows`)	Complex (plugins)	Simple (`.gitlab-ci.yml`)
Auto-Trigger	Yes	Yes	Yes
Docker Support	Native	Via plugins	Integrated
Interface	Integrated GitHub	Blue Ocean / plugins	Integrated GitLab
Maintenance	Very low	High	Low to moderate
Community	Growing rapidly	Large	Active

Table 2.9: Comparison of Main Continuous Integration (CI) Tools

For this project, **GitHub Actions** was chosen to automate the integration and deployment pipeline. Because BeBouncy's application repositories are already hosted on GitHub, this solution enables seamless end-to-end automation from tests and code analysis through Docker image builds and pushes to GHCR directly via `.github/workflows` YAML files, without any additional server infrastructure.

2.4.7 Continuous Deployment (CD) Tools in a GitOps Approach

Continuous Deployment (CD) is a central component of DevSecOps and GitOps practices, as it enables the automated and reliable delivery of applications to Kubernetes environments. In a GitOps model, deployment operations are driven directly from Git repositories, which serve as the single source of truth for infrastructure and application configurations. Selecting an appropriate CD solution is therefore essential to ensure deployment consistency, traceability, security, and synchronization between the declared and actual states of the cluster.

To identify the most suitable solution for the project, several deployment tools were evaluated according to criteria such as Kubernetes integration, GitOps compatibility, deployment automation capabilities, observability, security features, and ease of configuration and maintenance.

Criterion	**Argo CD**	**Jenkins**	**FluxCD**
Tool Type	Specialized GitOps	Versatile CI/CD platform	Lightweight GitOps tool
Deployment Strategy	Pull-based (Git-driven)	Push-based (manual or pipeline-triggered)	Pull-based (Git state tracking)
User Interface	Intuitive visual web UI	Customizable via plugins	Minimalist CLI interface
Kubernetes Integration	Native and optimized	Via additional plugins	Native and fluid
Configuration Management	Helm, Kustomize, YAML	Jenkinsfile + plugins	Helm, Kustomize, YAML
Security Features	RBAC, Git audit, manifest signing	Depends on manual configuration	Basic RBAC; fewer controls than ArgoCD
Observability	Real-time sync status view	Pipeline logs, plugins required	Less visual; logs and CLI
Complexity	Simple installation and configuration	Higher learning curve	Simple and fast configuration
GitOps Popularity	One of the current GitOps leaders	Poorly suited to pure GitOps	Widely used in the CNCF ecosystem

Table 2.10: Comparison of Continuous Deployment (CD) Tools

For deployment management on the Kubernetes cluster, **ArgoCD** was chosen for its native GitOps approach, which enables continuous and automatic synchronization between the desired state declared in the `BB-INFRA` repository and the actual state of the cluster. ArgoCD's intuitive visual interface allows real-time tracking of deployments and resource conformity, greatly simplifying environment management and control.

2.4.8 Infrastructure as Code (IaC) Tools

IaC (Infrastructure as Code) is central to DevOps, as it enables environments to be automated and managed through code, ensuring consistency, reproducibility, and speed of deployments while reducing human error.

In this project, all Kubernetes resources are defined using **Helm chart templates** parameterized through environment-specific configuration files (`values-qa.yaml` and `values-prod.yaml`) stored in the `BB-INFRA` repository. This design ensures that the complete cluster state remains version-controlled, deterministic, and fully reproducible directly from Git, which aligns with the core principles of the GitOps methodology.

For virtual machine provisioning and the initial cluster bootstrap process, **shell scripts** were used to automate the installation and configuration of Kubernetes on the OVHcloud virtual machines. Full provisioning frameworks such as Terraform were intentionally not adopted for this layer because the infrastructure scope remained relatively small, the bootstrap process required rapid iteration during experimentation, and shell scripting provided a lightweight solution that aligned with the team's existing operational practices.

2.4.9 Kubernetes Deployment Packages

To deploy applications and services within Kubernetes, several approaches are available, each offering specific functionalities for management, configuration, and deployment automation. The choice depends on the project's complexity, reuse requirements, and CI/CD integration practices.

Criterion	YAML Manifests	Kustomize	Helm Charts
Simplicity	YAML files applied directly	Adds `kustomization.yaml` for overlays	Helm syntax and templates; more complex
Customization	Modify or duplicate YAML per environment	Overlays for different environments	Configurable parameters via `values.yaml`
Reusability	Low; frequent duplication	Good with base + overlays	Excellent; packaged and distributable charts
Version Management	No native versioning	Versioning via Git	Integrated versioning; rollback possible
Required Tools	Only `kubectl`	Integrated in `kubectl apply -k`	Helm installed on the cluster
Ideal Use Case	Small projects, quick tests	Medium projects with multiple environments	Complex projects requiring packaging and CI/CD

Table 2.11: Compact Comparison of Kubernetes Deployment Packages

In this project, **Helm Charts** were chosen for deploying all BeBouncy services and the monitoring stack (Prometheus and Grafana), because this approach offers rapid, versioned, and easily configurable deployments, perfectly suited to a complex environment requiring automation and CI/CD integration. The use of parameterized `values.yaml` files per environment (QA / production) also enables clean environment promotion without manifest duplication.

2.4.10 Visualization and Dashboard Tools

Visualization and dashboard tools play a crucial role in modern monitoring and observability practices by transforming raw operational data into clear and actionable insights. Through centralized dashboards and graphical representations of metrics, these tools facilitate the real-time monitoring of infrastructure and application health, support rapid anomaly detection, and assist teams in tracking key performance indicators. In the context of DevSecOps and cloud-native environments, selecting an appropriate

visualization solution is essential to ensure efficient observability, incident response, and operational decision-making.

Criterion	**Grafana**	**Tableau**	**Kibana**
Tool Type	Dashboard & monitoring	BI / Reporting	Log analysis & dashboards
Data Sources	Prometheus, InfluxDB, MySQL, Elasticsearch	Relational databases, CSV, cloud	Elasticsearch
Integrated Alerting	Yes	No	No
Ease of Use	Very good	Very good	Medium
Scalability	High	Medium	High
Cost	Open source	Paid	Open source

Table 2.12: Comparison of Visualization Tools

Grafana was retained for its ability to manage real-time metrics, its native integration with Prometheus, and its built-in alerting capabilities.

2.4.11 Metrics Monitoring Tools

Metrics monitoring is a fundamental component of infrastructure observability and system reliability. Monitoring tools enable the continuous collection, storage, and analysis of technical and operational metrics related to applications, containers, and cluster resources. These solutions provide visibility into system performance, facilitate the early detection of anomalies, and support proactive incident management through alerting mechanisms. In Kubernetes-based environments, choosing an efficient monitoring platform is essential to guarantee service availability, scalability, and operational stability.

Criterion	Prometheus	Zabbix	Datadog
Monitoring Type	Metrics / TSDB	Infrastructure & metrics	Cloud monitoring
Data Collection	Pull	Pull / Agent	Agent / API
Alerting	Yes (via Alertmanager)	Yes	Yes
Installation Ease	Medium	Medium	Easy
Scalability	Medium	Medium	High
Cost	Open source	Open source	Paid

Table 2.13: Comparison of Monitoring Tools

Prometheus was chosen for its strength in real-time metrics collection and its native integration with Kubernetes and Alertmanager.

2.4.12 Alert Management Tools

Alert management constitutes an essential aspect of monitoring and incident response strategies within DevSecOps environments. By processing and routing alerts generated by monitoring systems, alert management tools help ensure rapid reaction to operational issues, minimize service interruptions, and maintain infrastructure reliability. Effective alerting mechanisms also reduce noise through alert grouping and deduplication, allowing teams to focus on critical incidents and improve response efficiency. In Kubernetes-based infrastructures, selecting an appropriate alert management solution is particularly important to guarantee continuous service availability and proactive operational supervision.

Criterion	Alertmanager	PagerDuty	Opsgenie
Alert Management	Centralized	Centralized	Centralized
Multi-channel Notifications	Email, Slack, Webhook	Email, SMS, Slack	Email, SMS, Slack
Suppression / Grouping	Yes	Yes	Yes
Integration	Prometheus	Various tools	Various tools
Configuration Ease	Medium	Very good	Very good
Cost	Open source	Paid	Paid

Table 2.14: Comparison of Alert Management Tools

Alertmanager is preferred for Prometheus alert management thanks to its zero cost, and its features for alert grouping, deduplication, and routing to the BeBouncy team via Slack.

2.5 Security Policies and Access Management

Security is a fundamental axis in the implementation of a modern infrastructure. It concerns not only data protection, but also the guarantee of effective governance of accesses, secrets, and container images used in production environments. This section presents the main security practices applied in the context of this project.

2.5.1 Access Management and Authentication

In order to reduce the risks associated with intrusions or privilege abuse, the infrastructure relies on several access management strategies:

- **Principle of Least Privilege:** every user, service, or process is granted only the rights strictly necessary for its role, thereby limiting the blast radius in the event of a compromise.
- **RBAC (Role-Based Access Control):** permission management in Kubernetes is organized according to roles (`Role`, `ClusterRole`) applied to users or service accounts via bindings (`RoleBinding`, `ClusterRoleBinding`).
- **Token-based authentication:** access to tools (GitHub Actions, Kubernetes, image registries)

is secured through tokens and secrets, enabling each automated action to be controlled and audited.

- **TLS (Transport Layer Security):** communications between cluster components (API server, kubelet, nodes, etc.) are encrypted to prevent interception or tampering.

2.5.2 Secure Secret Management

The protection of sensitive information (passwords, API keys, certificates) rests on several practices:

- **HashiCorp Vault:** centralized storage and dynamic delivery of secrets to applications at runtime, preventing plaintext secrets from ever appearing in repository files.
- **Kubernetes Secrets:** secure storage and distribution of confidential information to pods within the cluster.
- **Controlled mounting:** secrets are injected only into the containers that require them, using volume mounts or environment variable injection.
- **Regular key rotation:** periodic replacement of tokens and passwords to reduce the risk of long-term compromise.

2.5.3 Docker Image Security

Security also extends to the management of containerized images:

- **Source verification:** use of official images from trusted registries (Docker Hub official images, GHCR).
- **Vulnerability scanning:** images are analyzed with Trivy at every pipeline execution to identify known CVEs before any artifact reaches the registry.
- **Security updates at build time:** application of OS-level security patches during image construction.
- **Attack surface minimization:** construction of lean, multi-stage images containing only the strictly necessary libraries and runtime dependencies, without development tooling.

2.5.4 Monitoring and Compliance

Finally, security is maintained throughout the entire project lifecycle:

- **Audit and traceability:** all deployments are traceable to a specific Git commit in `BB-INFRA`, providing a full audit trail of who changed what and when.
- **Access and anomaly monitoring:** Grafana dashboards and Alertmanager rules are configured to raise alerts on unauthorized access attempts or suspicious resource usage patterns.
- **Continuous compliance:** regular evaluation of practices against DevSecOps standards to guarantee adaptation to emerging threats, supported by Trivy scans on every pipeline run.

Conclusion

In this chapter, we presented the state of the art as well as the architecture retained for the BeBouncy DevSecOps project. The analysis and comparison of various existing approaches allowed us to identify the most appropriate solution, combining robustness, performance, and scalability. The systematic justification of each technological choice from GitHub Actions and Docker to Kubernetes, ArgoCD, Trivy, SonarQube, Prometheus, Grafana, Vault, and Helm which establishes a coherent and well-grounded foundation for the practical implementation carried out in the following chapters.

Chapter 3

Sprint Release 1: Containerization, Kubernetes Orchestration, and GitOps CI/CD

Contents

Introduction

The first release of the *BeBouncy* DevSecOps internship project lays the architectural foundation upon which every subsequent phase is built. Prior to this release, the platform's four core services **API**, **Web**, **AI**, and **Microservices** were managed as bare Node.js processes supervised by PM2 directly on virtual machines. Deployments relied on manual SSH sessions, environment variables were stored in plaintext `.env` files, and there was no enforced parity between the QA and production environments.

This release addresses those challenges through three interrelated workstreams:

1. **Dockerization**: packaging each service into optimised, hardened, production-grade container images.
2. **Kubernetes orchestration on the QA node**: deploying and managing those images inside a single-node Kubernetes cluster, governed by an Infrastructure-as-Code (IaC) repository using Helm.
3. **GitOps CI/CD**: establishing fully automated build, push, and deployment pipelines via GitHub Actions and ArgoCD so that every code commit to the `main` branch results in a traceable, auditable deployment to QA.

The chapter begins with an assessment of the existing infrastructure, motivates the architectural decisions taken, and then details the implementation of each workstream in sequence, concluding with the end-to-end GitOps workflow and the rollback strategy that completes the release.

3.1 Existing Infrastructure and Motivation

3.1.1 Pre-existing Architecture

Table 3.1 summarises the infrastructure state at the start of the release.

Component	**Current Setup**
QA Server	(b2-7-rbx-a, OVH)
Production Server	(b3-8, OVH)
Process Management	PM2 on both servers
Deployment Method	GitHub Actions SSH + PM2 restart commands
Repositories	API, Microservices, Web, AI

Table 3.1: Pre-existing Infrastructure Overview

All four repositories ran Node.js 24 (except *Web*, which was on Node.js 18/20). Each service was installed and started individually on the host OS, with no isolation between them. A failure or

resource spike in one service could directly impact the others.

3.1.2 Motivation for Containerisation

Before the adoption of Docker and Kubernetes, the application stack was deployed directly on VPS instances using PM2. While this approach was initially sufficient for rapid development, it became increasingly difficult to maintain as the infrastructure evolved. Repeated operational issues highlighted the limitations of manual deployments and mutable server environments.

3.1.2.1 Before vs. After Containerisation

Challenge	**Before (PM2 on VPS)**	**After (Docker + Kubernetes)**
Deployment	Manual SSH commands executed on servers for every release, increasing the risk of human error	Automated deployment pipeline triggered directly from Git pushes
Environment consistency	QA and production environments frequently differed in Node.js versions and system packages	The same immutable container image runs identically across all environments
Rollback process	Rollback required manual redeployment of older application versions	Instant rollback using Kubernetes rollout history and revision control
Secrets management	Sensitive `.env` files stored on servers and occasionally committed to repositories	Centralized secret injection using Kubernetes Secrets and Vault
Scaling	Scaling required manual PM2 instance management and service restarts	Horizontal scaling handled automatically through Kubernetes replicas and HPA
Resource isolation	Applications competed for CPU and memory resources on the host	Containers enforce CPU and memory limits to isolate workloads
Failure recovery	PM2 could restart processes, but host failures required manual intervention	Kubernetes automatically reschedules failed pods onto healthy nodes

Challenge	Before (PM2 on VPS)	After (Docker + Kubernetes)
Service discovery	Services relied on hardcoded IP addresses and manual load balancer configuration	Built-in Kubernetes DNS enables automatic service discovery
Zero-downtime updates	PM2 reload mode reduced downtime but could still interrupt traffic	Rolling updates gradually replace pods without service interruption
Configuration drift	Servers accumulated unique manual changes over time	Infrastructure state is versioned declaratively in Git
Disk usage	Each service maintained separate local dependencies and runtime files	Container layers are shared and optimized through image caching

Table 3.2: Before and After Containerisation Comparison

The transition from PM2-based deployments to a containerized orchestration platform fundamentally changed how infrastructure and applications were managed. Docker introduced immutable and reproducible runtime environments, while Kubernetes added orchestration, resilience, and scalability capabilities that were previously unavailable.

3.2 Phase 1: Dockerisation

3.2.1 Design Principles

Three principles governed the containerisation strategy:

Single-Responsibility Containers Each container runs exactly one process. The *Microservices* repository, for example, separates its scheduler and worker concerns using an entrypoint script controlled by an environment variable (`SCHEDULER=1`), allowing the same image to fulfill two roles without duplication.

Ephemeral and Stateless Containers All persistent state resides outside the container: MongoDB for data, Redis for sessions and caching, and OVH Object Storage for binary assets. Containers can be destroyed and re-created at any time without data loss.

Immutable Infrastructure A running container is never modified. Updates are performed by building a new image and replacing the container, an approach that eliminates configuration

drift and enables instant rollback.

3.2.2 Multi-Stage Dockerfile Design

Every repository uses a two-stage build to produce a minimal, hardened production image.

Stage 1: Builder

The builder stage uses the full `node:24-alpine` image, which includes compiler toolchains required by native modules (e.g., `python3`, `make`, `g++`). This stage:

1. Copies `package.json` and the lock file.
2. Installs *all* dependencies (including `devDependencies`).
3. Copies the application source code.
4. Executes any build steps (TypeScript compilation, Next.js static generation, etc.).

Stage 2: Production Runner

The production stage starts from a fresh `node:24-alpine` base discarding the entire builder layer and:

1. Installs only the lightweight runtime utility `dumb-init` (for correct signal propagation and zombie-process reaping).
2. Creates a dedicated non-root system group (`nodejs`, GID 1001) and user (`appuser`, UID 1001).
3. Copies only the production `node_modules` and built application artefacts from the builder stage, using `-chown` to set correct ownership.
4. Configures file-system permissions for the `/app/logs` directory.
5. Defines a `HEALTHCHECK` instruction that polls the service's `/health` HTTP endpoint.
6. Sets `ENTRYPOINT ["dumb-init", "-"]` and an appropriate `CMD`.

The result is a final image that contains *only what is needed to run the service* no build tools, no source-map files, no `devDependencies`.

Security Hardening Applied to All Dockerfiles

- **Non-root execution**: `USER appuser` ensures the process never runs as root, limiting the blast radius of a container escape.

- **Minimal base image**: `node:24-alpine` has fewer OS packages than Debian-based images, drastically reducing the CVE surface area.
- **Explicit signal handling**: `dumb-init` acts as PID 1 and correctly forwards SIGTERM to the Node.js process, enabling graceful shutdown.
- **Build-context exclusion**: A comprehensive `.dockerignore` file prevents `node_modules`, `.env` files, test artefacts, IDE metadata, and CI configuration from being included in the build context, reducing both image size and the risk of accidental secret leakage.
- **Environment variable hygiene**: `HUSKY=0` prevents Git hook tooling from executing inside the container build environment.

Microservices Entrypoint Strategy

The *Microservices* repository contains both a scheduler (`bootstrap.js`) and an Agenda.js worker (`agenda-worker.js`). Rather than maintaining two separate Dockerfiles, a shell entrypoint script (`docker-entrypoint.sh`) selects the process to launch based on the `SCHEDULER` environment variable

This pattern keeps the image layer count low and ensures that both roles are tested against the same artefact.

3.2.3 Image Optimisation Results

The multi-stage build, Alpine base image, and build-context exclusions produced a 70–80% reduction in image size compared to a naïve single-stage build on a full Debian Node image. Images in the range of 180–300 MB replaced earlier prototypes exceeding 800 MB–1.5 GB. Smaller images translate to faster CI/CD pipeline runs, reduced GHCR storage consumption, and a smaller attack surface.

Figure 3.1: Docker images before optimization

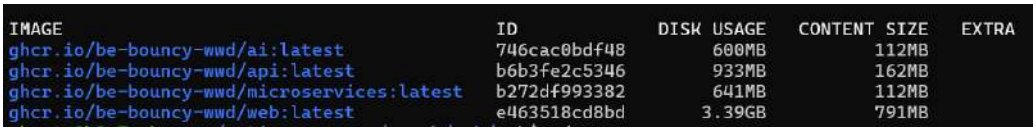


Figure 3.2: Docker images after optimization

Although image optimization techniques such as multi-stage builds and layer caching were applied, the Web image size remained high due to the substantial dependency footprint required by

the application and its associated tooling. GHCR-stored images are further compressed at the layer level and therefore appear smaller than local `docker images` output.

3.2.4 GitHub Container Registry (GHCR)

Built images are pushed to the GitHub Container Registry (`ghcr.io`), organised as:

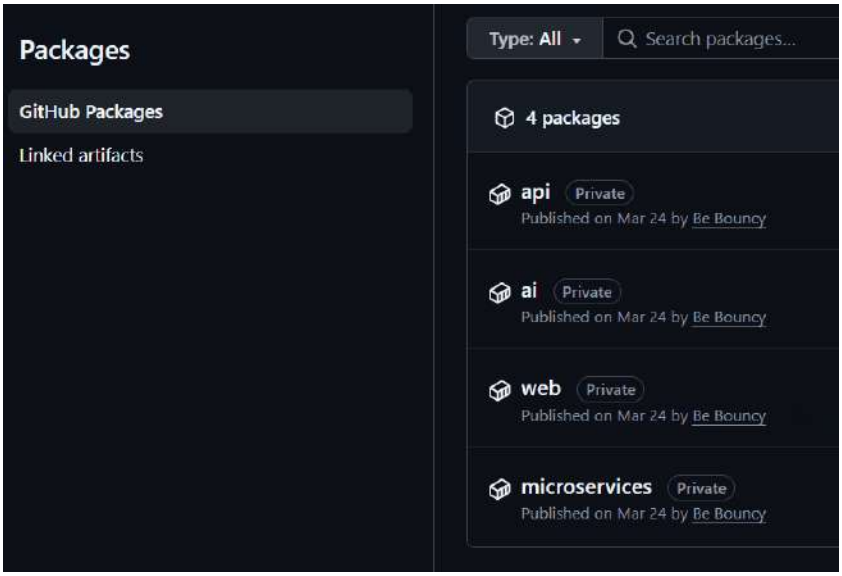


Figure 3.3: GitHub Container Registry

GHCR was chosen over alternatives because it integrates natively with GitHub Actions (authentication via `GITHUB_TOKEN`), links packages directly to their source repositories, and is free for private repositories within the organisation's plan.

Image Tagging Strategy

Each push produces multiple tags to support different use cases:

Tag Type	Format	Purpose
Short SHA	`sha-a1b2c3d`	Immutable reference to exact commit
Branch	`main`	Latest build from the main branch
Latest	`latest`	Alias updated on every successful main push
Semantic	`v1.2.3`	Release versions (from `package.json`)

Table 3.3: Image Tagging Convention

For QA deployments, the short commit SHA tag is used exclusively in the Helm values files, ensuring complete traceability between what runs in the cluster and which commit produced it.

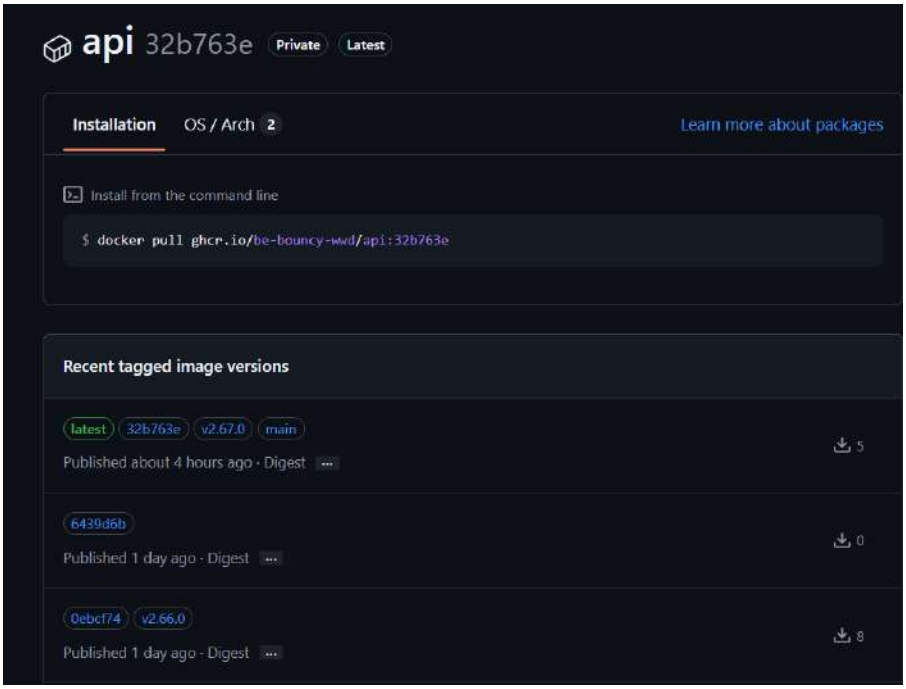


Figure 3.4: API images stored in GHCR

3.3 Phase 2: Kubernetes Orchestration on the QA Node

3.3.1 Cluster Setup

A single-node Kubernetes cluster was provisioned on the QA virtual machine. The cluster was installed using a lightweight, CNCF-conformant Kubernetes distribution well-suited to resource-constrained environments. Post-installation, `kubectl` access was configured for the non-root user by adjusting ownership of the kubeconfig at `/etc/rancher/k3s/k3s.yaml` and persisting the `KUBECONFIG` environment variable in the user's shell profile.

The cluster topology for the QA environment is:

```
Cluster (QA)
 +-- Node: b2-7-rbx-a
        +-- Namespace: qa
                +-- Pod: api-qa           (port 3005)
                +-- Pod: web-qa           (port 4000)
                +-- Pod: ai-qa            (port 3006)
                +-- Pod: microservices-qa (port 3007)
```

3.3.2 Ingress Controller Selection and Configuration

The selected Kubernetes distribution includes **Traefik** as the default Ingress Controller for exposing HTTP and HTTPS services. However, the QA environment already relied on an existing **Nginx**

reverse proxy architecture integrated with **Let's Encrypt** certificates managed through Certbot for secure HTTPS communication on `qa.bebouncy`.

An evaluation of both solutions led to retaining Nginx as the primary external entry point. This decision was motivated by Nginx's mature ecosystem, extensive industry adoption, advanced reverse proxy features, and strong compatibility with the existing SSL certificate management workflow already established within the infrastructure. Preserving the Nginx layer also ensured architectural consistency and reduced unnecessary operational complexity.

Consequently, Traefik was disabled in the Kubernetes server configuration using the `-disable=traefik` flag, while Nginx continued to manage external traffic routing, HTTPS termination, and certificate renewal. In the final architecture, Kubernetes *Ingress* resources are used solely for internal service routing within the cluster.

3.3.3 Namespace Strategy

Two namespaces were created:

- `argocd` — houses the ArgoCD control-plane components.
- `qa` — houses all application workloads. The `CreateNamespace=true` ArgoCD sync option means ArgoCD itself creates and labels the `qa` namespace on first sync, reducing manual bootstrapping steps.

```
ubuntu@b2-7-rbx-a:~$ kubectl get pods -n qa
NAME                                READY   STATUS    RESTARTS      AGE
ai-qa-7875bbd546-pdlb7              1/1     Running   0             6h7m
api-qa-84f8bf7db7-l9d4s             1/1     Running   0             5h54m
microservices-qa-657bf7b686-ht8gs   1/1     Running   1 (10h ago)   3d7h
web-qa-567c5898f6-xqnmt             1/1     Running   0             22m
ubuntu@b2-7-rbx-a:~$ kubectl get deployments -n qa
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
ai-qa             1/1     1            1           69d
api-qa            1/1     1            1           68d
microservices-qa  1/1     1            1           55d
web-qa            1/1     1            1           69d
ubuntu@b2-7-rbx-a:~$ kubectl get svc -n qa
NAME              TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
ai-qa             ClusterIP   10.43.139.225   <none>        3006/TCP   69d
api-qa            ClusterIP   10.43.197.249   <none>        3005/TCP   68d
microservices-qa  ClusterIP   10.43.150.51    <none>        3007/TCP   69d
web-qa            ClusterIP   10.43.84.86     <none>        4000/TCP   69d
```

Figure 3.5: State of Pods, Deployments, and Services in the QA namespace

3.4 Phase 3: Infrastructure as Code with Helm

3.4.1 The Centralized Infrastructure Repository

A dedicated infrastructure repository, **BB-INFRA**, was created to centralise all Kubernetes manifests, Helm charts, and environment-specific configuration. Storing infrastructure definitions inside application

repositories would couple deployment lifecycles to code lifecycles, making it impossible to redeploy a service without touching its source repository and vice versa. BB-INFRA enforces a clean separation:

Application repositories own the code and the container image.

BB-INFRA owns the cluster state.

3.4.2 Helm as the IaC Package Manager

Helm is the package manager for Kubernetes. It uses *charts*—parameterised bundles of Kubernetes YAML templates—to define, install, and upgrade applications. The key advantage over raw manifests is *templating*: a single `deployment.yaml` template is re-used for all four services by substituting values from an environment-specific `values-*.yaml` file. This eliminates copy-paste drift between services and environments.

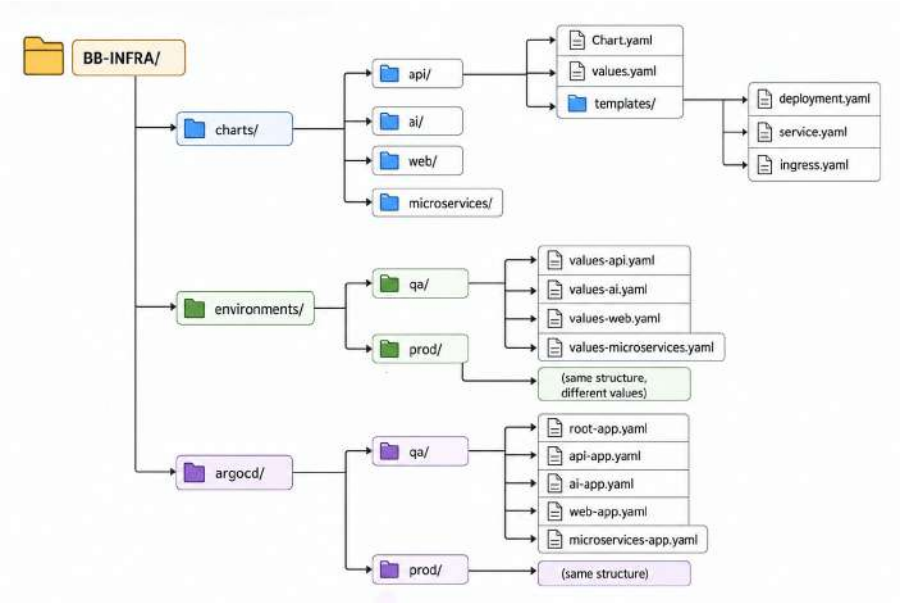


Figure 3.6: BB-INFRA Repository Structure

For example, deploying the API to QA requires only:

```
./charts/api --values ./environments/qa/values-api.yaml
```

```yaml
replicaCount: 1

image:
  repository: ghcr.io/be-bouncy-wwd/api
  tag: "224dc18"
  pullPolicy: Always

imagePullSecrets:
  - name: ghcr-secret

service:
  type: ClusterIP
  port: 3005

ingress:
  enabled: true
  className: "nginx"
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
  hosts:
    - host: api.qa.bebouncy.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - hosts:
        - api.qa.bebouncy.com
      secretName: api-qa-tls

resources:
  limits:
    cpu: 300m
    memory: 384Mi
  requests:
    cpu: 150m
    memory: 192Mi

autoscaling:
  enabled: false

config:
  env: qa
  logLevel: debug

envFrom:
  - secretRef:
      name: api-secrets

httpRoute:
  enabled: false
```

Figure 3.7: QA-specific values for API

The `values-api.yaml` file shown in Figure 3.7 contains the QA-specific configuration for the API service. Helm injects these values into reusable chart templates during deployment, allowing the same Kubernetes manifests to be reused across environments while only the configuration changes.

The file defines the container image and tag, replica count, resource limits, ingress configuration,

and environment-specific settings such as logging level. HTTPS exposure is configured through the NGINX Ingress Controller with automatic TLS certificate provisioning via Cert-Manager and Let's Encrypt.

Sensitive values are not stored directly in the repository; instead, they are loaded from Kubernetes Secrets using `envFrom.secretRef`. Together with ArgoCD, this enables a declarative GitOps workflow where updating the values file (e.g., changing the image tag) automatically synchronises the deployment in the Kubernetes cluster.

3.5 Phase 4: GitOps Continuous Delivery with ArgoCD

3.5.1 Adopting GitOps with ArgoCD

GitOps approach allows us to make deployments declarative, traceable, and automatically synchronised with the cluster state.

GitOps treats **Git as the single source of truth**: the desired infrastructure and application configuration are stored in BB-INFRA, while a controller continuously reconciles the live Kubernetes cluster against that declared state. Any change committed to Git is automatically applied to the cluster, while configuration drift caused by manual modifications is detected and corrected.

ArgoCD was selected as the GitOps controller due to its native Kubernetes integration, declarative deployment model, automatic synchronisation capabilities, and intuitive web interface for monitoring application health and deployment status. It also integrates naturally with the Helm-based infrastructure repository.

ArgoCD was installed in a dedicated `argocd` namespace on the QA Kubernetes cluster.

```
ubuntu@b2-7-rbx-a:~$ kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS      AGE
argocd-application-controller-0                     1/1     Running   1 (10h ago)   3d7h
argocd-dex-server-5f6bcb7ff9-dgqxq                  1/1     Running   1 (10h ago)   3d7h
argocd-notifications-controller-7c77c56dd8-j7wz7    1/1     Running   1 (10h ago)   3d7h
argocd-redis-59bfbdf575-hvjw9                       1/1     Running   1 (10h ago)   3d7h
argocd-repo-server-54cf65c9d4-nd8rz                 1/1     Running   0             5h24m
argocd-server-8ff4b9795-8wzng                       1/1     Running   1 (10h ago)   3d7h
ubuntu@b2-7-rbx-a:~$ kubectl get deployments -n argocd
NAME                               READY   UP-TO-DATE   AVAILABLE   AGE
argocd-dex-server                  1/1     1            1           74d
argocd-notifications-controller    1/1     1            1           74d
argocd-redis                       1/1     1            1           74d
argocd-repo-server                 1/1     1            1           74d
argocd-server                      1/1     1            1           74d
ubuntu@b2-7-rbx-a:~$ kubectl get svc -n argocd
NAME                                       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
argocd-applicationset-controller           ClusterIP   10.43.37.45      <none>        7000/TCP,8080/TCP            74d
argocd-dex-server                          ClusterIP   10.43.18.29      <none>        5556/TCP,5557/TCP,5558/TCP   74d
argocd-metrics                             ClusterIP   10.43.21.233     <none>        8082/TCP                     74d
argocd-notifications-controller-metrics    ClusterIP   10.43.13.12      <none>        9001/TCP                     74d
argocd-redis                               ClusterIP   10.43.207.44     <none>        6379/TCP                     74d
argocd-repo-server                         ClusterIP   10.43.118.156    <none>        8081/TCP,8084/TCP            74d
argocd-server                              ClusterIP   10.43.8.110      <none>        80/TCP,443/TCP               74d
argocd-server-metrics                      ClusterIP   10.43.234.123    <none>        8083/TCP                     74d
```

Figure 3.8: State of pods, deployments and services in ArgoCD namespace

The initial administrator password was retrieved from the bootstrap secret and changed immediately after installation. Access to the ArgoCD web interface is provided locally through `kubectl port-forward`,

with SSH tunnelling used when remote access is required.

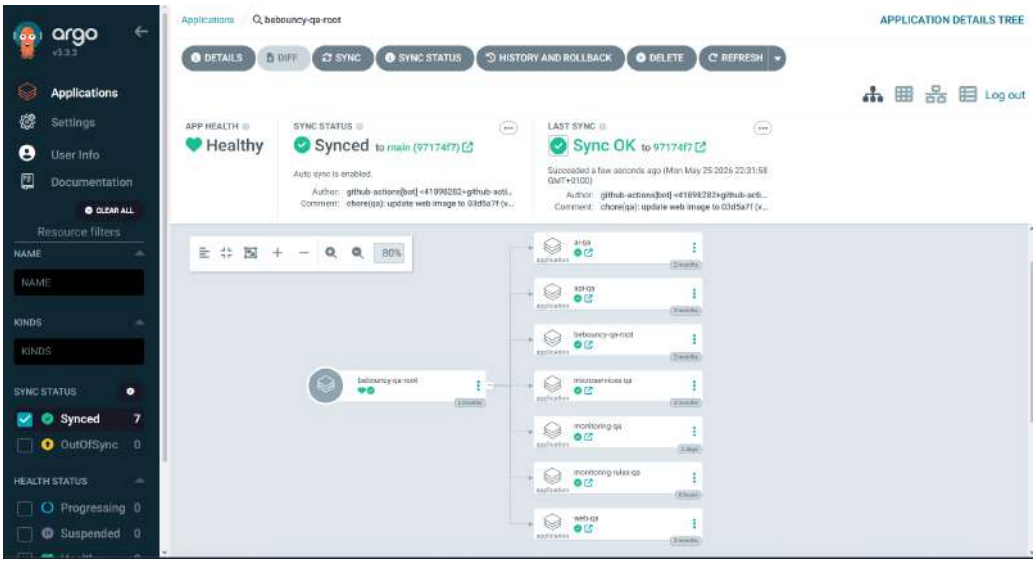


Figure 3.9: ArgoCD Web UI

3.5.2 App of Apps Pattern

BB-INFRA implements the **App of Apps** pattern: a *root application* ArgoCD manifest, residing at `argocd/qa/root-app.yaml`, is applied manually once during bootstrapping. That root application then creates and manages *child* ArgoCD `Application` resources one per service,each of which points to the corresponding Helm chart and QA values file.

```
ArgoCD Root App (argocd/qa/root-app.yaml)
  +-- Child App: api-qa
  +-- Child App: web-qa
  +-- Child App: ai-qa
  +-- Child App: microservices-qa
```

This structure provides:

- **Single-command bootstrap**: Apply the root app once; everything else is managed declaratively.
- **Granular visibility**: Each service's health and sync status is independently observable in the ArgoCD UI.
- **Independent rollback**: A single service can be rolled back without affecting the others.

3.5.3 ArgoCD Sync Configuration

Each child application is configured with `automated` sync policy, which includes:

- `prune: true` — Kubernetes resources removed from the Git manifest are automatically deleted from the cluster.
- `selfHeal: true` — Any manual change applied directly to the cluster (e.g., via `kubectl`) is reverted to match Git within the next polling cycle.
- `CreateNamespace: true` — The `qa` namespace is created automatically if it does not exist.

ArgoCD polls BB-INFRA every three minutes by default. In practice, a commit to BB-INFRA triggers a deployment within that window, without any manual intervention.

Figure 3.10: ArgoCD's automated syncing

3.5.4 How ArgoCD Consumes Helm

ArgoCD does not simply apply raw YAML. When it detects a change in BB-INFRA, it executes the equivalent of `helm template` against the chart and the applicable `values-*.yaml` file, produces the rendered Kubernetes manifests, diffs them against the live cluster state, and applies only the changed resources. This means the full expressiveness of Helm templating is available while ArgoCD retains complete visibility into what will be applied.

With the GitOps delivery model established through ArgoCD and Helm, the next step was automating the application lifecycle itself. This was achieved by implementing a CI/CD pipeline using GitHub Actions, responsible for validating code, building container images, publishing them to the registry, and updating the GitOps repository to trigger automated deployments.

3.6 Phase 5: GitHub Actions CI/CD Pipeline

3.6.1 Pipeline Overview

Each of the four application repositories contains a GitHub Actions workflow file (`auto-build-and-push.yml`). The workflow is triggered on every push to the `main` branch or via manual dispatch (`workflow_dispatch`).
It is structured as three sequential jobs:

Job 1: Security Scan Source → Job 2: Build, Test & Push → Job 3: Update BB-INFRA

Each job declares a `needs:` dependency on the preceding one, ensuring the pipeline fails fast at the earliest failure point.

3.6.2 Job 1 - Security Gate: Source Code Scan

Before any image is built, the pipeline performs a comprehensive scan of the source code using **Trivy** in filesystem mode (`trivy fs`). Trivy inspects:

- **Dependency vulnerabilities**: Known CVEs in `package-lock.json` or `yarn.lock` entries.
- **Hardcoded secrets**: API keys, tokens, and credentials accidentally present in the codebase.
- **Misconfigurations**: Insecure patterns in Dockerfile and Kubernetes configuration fragments.

The scan is configured with `severity: CRITICAL,HIGH` and `exit-code: 1`. A `CRITICAL` or `HIGH` finding raises a pipeline failure, blocking the subsequent build job.

3.6.3 Job 2 - Build, Test, and Push

This job executes only if Job 1 succeeds. Its steps are:

1. **Checkout** (with full history via `fetch-depth: 0` for accurate SHA computation).
2. **Compute short SHA**: `GITHUB_SHA::7` is extracted and stored as an environment variable; this becomes the primary image tag.
3. **Install dependencies** using the lock-file-aware conditional: `yarn -frozen-lockfile` if `yarn.lock` is present, otherwise `npm ci`.
4. **Run tests**: The test step checks whether a `test` script exists in `package.json` before invoking it, preventing pipeline failure in repositories that do not yet have a test suite.
5. **Docker Buildx setup**: Enables BuildKit caching and multi-platform build capability.
6. **GHCR authentication**: Login using the `GHCR_PAT` repository secret.
7. **Metadata generation**: The `docker/metadata-action` generates all required tags (SHA, `latest`, semantic version).
8. **Build and push**: The `docker/build-push-action` builds the multi-stage image from the repository's Dockerfile and pushes it to GHCR with all generated tags. BuildKit layer caching is enabled for speed.

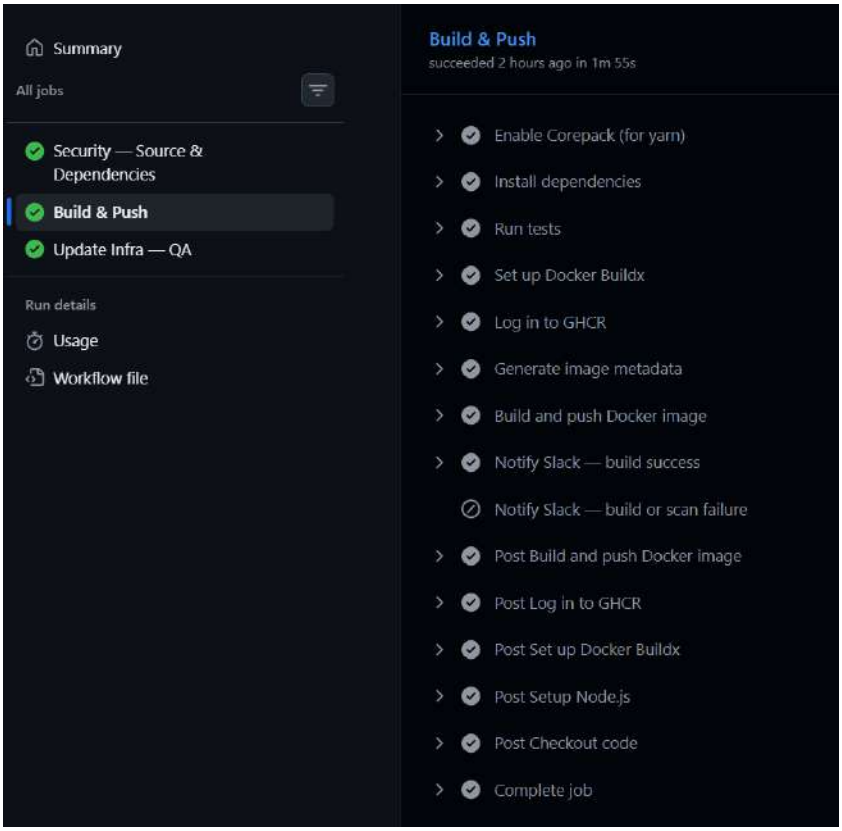


Figure 3.11: Build, Test and Push job

3.6.4 Job 3 - Update BB-INFRA (GitOps Trigger)

This job runs only on the QA environment path (push to `main` or manual QA dispatch) and only after Job 2 succeeds. It performs the critical GitOps handoff:

1. Checkout the **BB-INFRA** repository using a bot token with write access.
2. Locate the correct values file: `environments/qa/values-{service}.yaml`.
3. Replace the `tag` field with the new short SHA using an inline `sed` command.
4. Commit and push the change to BB-INFRA.

This single commit to BB-INFRA is the *only* thing the pipeline does to trigger a deployment. ArgoCD detects the new tag in its next polling cycle, re-renders the Helm chart, and issues a rolling update to the `qa` namespace. There are no SSH commands to servers, no `kubectl apply` calls from CI, and no cluster credentials stored in GitHub Secrets.

3.6.5 Slack Notifications

Slack webhook notifications are dispatched on success, and on failure of pipeline, providing the team with real-time visibility into the pipeline state without having to monitor the GitHub Actions UI.

3.6.6 Complete Pipeline Flow

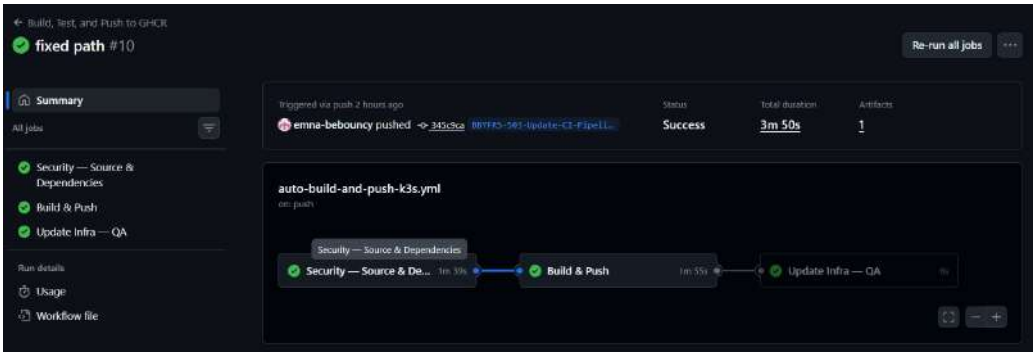


Figure 3.12: Pipeline flow

The automated pipeline therefore integrates the entire delivery chain, from source-code validation and image publication to GitOps-driven deployment updates. Since every deployment is versioned through immutable image tags stored in Git, the same workflow also enables a simple and reliable rollback mechanism.

3.7 Rollback Strategy

3.7.1 Design

Every deployed image is permanently addressable by its short SHA tag in GHCR. Because BB-INFRA is the single source of truth for the running tag, rolling back is equivalent to updating the values file to a previously deployed SHA, an operation that triggers the standard ArgoCD sync-and-deploy flow, with no special tooling required.

3.7.2 Rollback Pipeline

A dedicated `workflow_dispatch` pipeline (`rollback.yml`) was created for controlled rollbacks. It accepts the following inputs:

- `environment`: `qa` or `prod`.
- `target_tag`: The short SHA to roll back to.
- `confirm_rollback`: Must equal the string `"confirm"` when targeting production (an explicit safety gate against accidental rollbacks).

3.7.3 Rollback Execution Flow

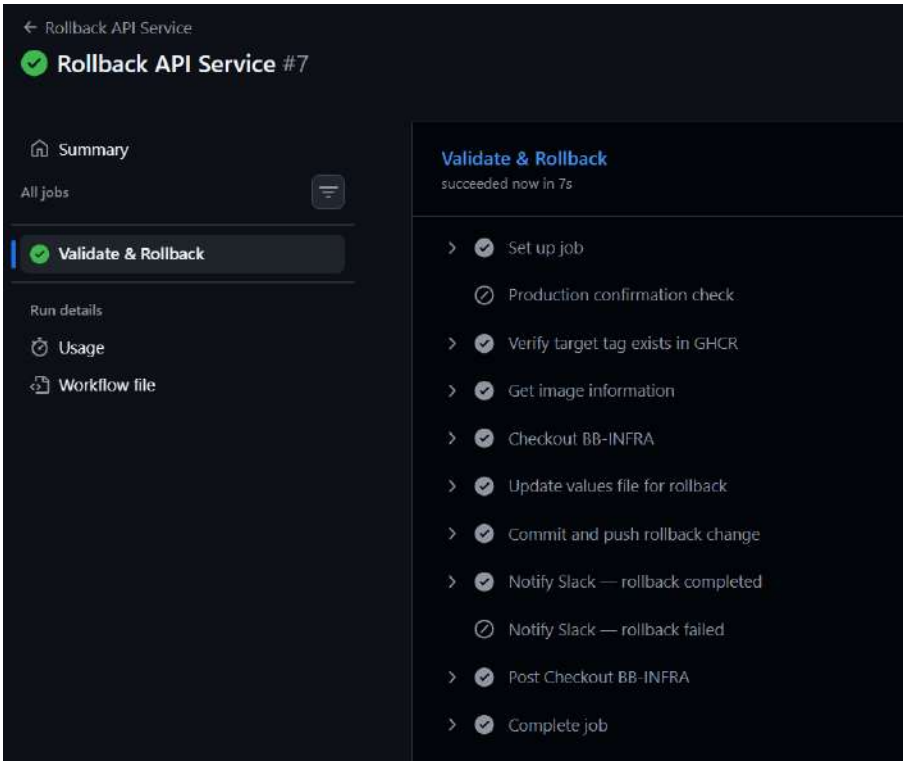


Figure 3.13: Rollback Pipeline steps

The pipeline skips the commit step if the values file already contains the requested tag, preventing unnecessary ArgoCD sync cycles. Slack notifications inform the team of rollback success, or failure.

Conclusion

This release transformed the BeBouncy platform from a manually operated collection of PM2 processes into a fully containerised, orchestrated, and GitOps-driven system. Four services were packaged into optimised, hardened Docker images, registered in GHCR, and deployed to a single-node Kubernetes cluster on the QA server. An Infrastructure-as-Code repository (BB-INFRA) governed by Helm charts and watched by ArgoCD became the single authoritative source of truth for the cluster state. A three-job GitHub Actions pipeline that automates the entire journey from code commit to running pod. A dedicated rollback pipeline completes the safety net by enabling rapid, validated reversion to any previously deployed image SHA.

With this foundation in place, the platform is ready for the security hardening phase addressed in the next release, which builds upon the pipeline, the Helm charts, and the cluster infrastructure established here to integrate SAST, DAST, IaC scanning, and comprehensive secrets management tooling.

Chapter 4

Sprint Release 2: DevSecOps Security Integration

Contents

Introduction

Release 1 established a fully automated containerisation and GitOps delivery pipeline. While that foundation included basic Trivy scans as a proof-of-concept security check, security was not yet a first-class architectural concern: the pipeline lacked structured static analysis, there was no validation of the Infrastructure-as-Code layer, no dynamic testing against the running application, and secret management was only partially implemented. The QA environment had also accumulated historical exposure risks in the form of plaintext credentials committed to version control.

Release 2 transforms the existing pipeline into a complete **DevSecOps** pipeline by embedding security controls at every stage of the software delivery lifecycle. The underlying philosophy is **Shift-Left Security**: rather than treating security as a final audit step before release, every meaningful security check is moved as early as possible in the pipeline, where the cost of remediation is lowest.

This release delivers five integrated security workstreams:

1. **SAST** - Static Application Security Testing via SonarCloud.
2. **Supply-Chain & Image Security** - Enhanced Trivy scanning with SBOM generation.
3. **IaC Security** - Infrastructure-as-Code validation via Checkov on Helm charts.
4. **DAST** - Dynamic Application Security Testing via OWASP ZAP post-deployment.
5. **Secret Management** - End-to-end secret lifecycle governance using HashiCorp Vault, Kubernetes Secrets, and Gitleaks-driven remediation.

Together, these workstreams implement a **defence-in-depth** model: no single control is relied upon exclusively; each layer catches classes of vulnerability that the others cannot.

4.1 DevSecOps Architecture Overview

4.1.1 Security Gates in the Delivery Pipeline

Figure 4.1 illustrates the position of each security control relative to the pipeline stages.

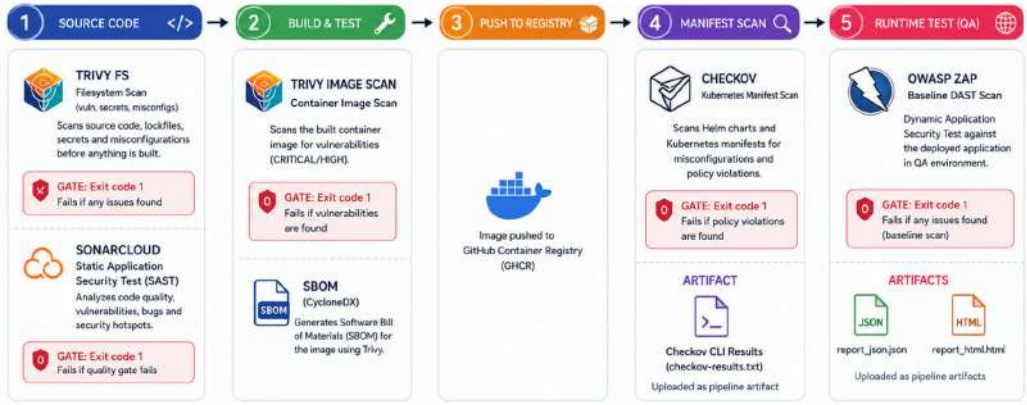


Figure 4.1: Security control in the Pipeline

Each gate is a *hard* or *soft* control:

- **Hard gate**: Pipeline fails and the deployment is blocked if the check does not pass. Applied to CRITICAL/HIGH vulnerability findings and SAST quality gates.
- **Soft gate**: Results are collected and reported as pipeline artefacts, flagged for review, but do not block the pipeline by default. Applied to certain informational Checkov findings and DAST low-severity findings. Thresholds can be tightened as the codebase matures.

4.1.2 Toolchain Summary

Tool	Category	Stage	Artefact Produced
Gitleaks	Secret scanning	Pre-pipeline	Console report, CI annotation
SonarCloud	SAST	Source	Quality Gate status, dashboard
Trivy FS	SCA / Misconfiguration	Pre-build	Console report
Trivy Image	Container CVE scan	Post-build	**SBOM (CycloneDX)**
Checkov	IaC security	Pre-deploy	Console report, JSON results
OWASP ZAP	DAST	Post-deploy	**HTML report**, **JSON report**
HashiCorp Vault	Secret management	Runtime	N/A (operational control)
Kubernetes Secrets	Secret injection	Runtime	N/A (operational control)

Table 4.1: DevSecOps Security Toolchain Summary

4.2 Static Application Security Testing - SonarCloud

4.2.1 What is SAST?

Static Application Security Testing analyses source code *without executing it.* A SAST tool parses the code into an abstract syntax tree (AST) or control-flow graph and applies a library of rules to detect patterns associated with security vulnerabilities, code quality defects, and maintainability issues.

Because SAST runs against source code before any build step, it is the earliest possible opportunity to detect logic-level security flaws.

4.2.2 Why SonarCloud?

SonarCloud is the cloud-hosted edition of SonarQube. It was selected for this project for the following reasons:

- **Native GitHub integration**: SonarCloud connects directly to the GitHub organisation, decorates pull requests with inline issue annotations, and reports Quality Gate status as a commit check.
- **Multi-language support**: The BeBouncy codebase spans JavaScript, TypeScript, and configuration formats; SonarCloud analyses all of these in a single scan.
- **Quality Gate enforcement**: A configurable Quality Gate defines pass/fail conditions; a failing gate blocks the pipeline at the SAST stage, before any image is built.

4.2.3 SonarCloud Integration in the Pipeline

The SonarCloud scan is executed in Job 1 of the pipeline, after dependency installation and before the Docker build. The scan is performed by the SonarCloud GitHub Action, which:

1. Authenticates to SonarCloud using a `SONAR_TOKEN` repository secret.
2. Uploads source code and, where available, test coverage reports to the SonarCloud analysis engine.
3. Waits for SonarCloud to return a **Quality Gate** verdict.
4. Fails the pipeline step (and therefore the job) if the Quality Gate status is `FAILED`.

Quality Gate Configuration

The Quality Gate was configured to enforce the following conditions on *new code* (code introduced since the previous analysis baseline):

What SonarCloud Detects in Node.js/TypeScript Projects

- **Injection vulnerabilities**: SQL injection, NoSQL injection, command injection patterns.
- **Authentication flaws**: Hardcoded credentials, weak hashing algorithms (MD5, SHA-1).
- **Sensitive data exposure**: Logging of sensitive variables, insecure cookie flags.

Metric	Condition	Effect on Failure
Security Rating	Worse than A	Hard gate -> pipeline fails
Reliability Rating	Worse than A	Hard gate -> pipeline fails
Maintainability Rating	Worse than A	Hard gate -> pipeline fails
Security Hotspots Reviewed	Less than 100%	Hard gate -> pipeline fails
Duplicated Lines (%)	Greater than 3%	Soft gate -> flagged only

Table 4.2: SonarCloud Quality Gate Conditions

- **Cross-site scripting (XSS)**: Unescaped user input rendered in HTTP responses.
- **Insecure dependencies**: Known-vulnerable package imports (complementary to Trivy).
- **Code smells and cognitive complexity**: Not security issues per se, but indicators of code that is likely to harbour bugs.

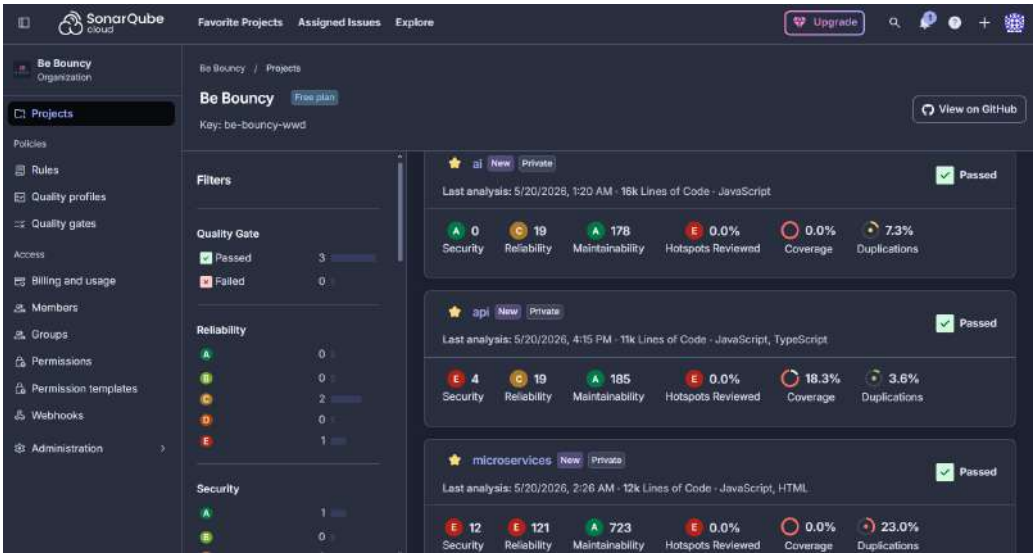


Figure 4.2: SonarCloud overview

4.3 Supply-Chain Security - Trivy Image Scanning

4.3.1 From Basic Check to Full Supply-Chain Audit

Release 1 introduced Trivy as a filesystem scanner for detecting vulnerabilities, secrets, and misconfigurations in the source code before build time. Release 2 extends this approach by adding post-build container image scanning with enforced security thresholds, structured reporting, and artefact generation. It also introduces **Software Bill of Materials (SBOM)** generation as a first-class pipeline output to improve software supply-chain visibility and traceability.

4.3.2 Trivy Image Scan and SBOM Generation

After the Docker image is built and pushed to GHCR, a dedicated Trivy image scan step runs against the registry image, ensuring that the scan result reflects the exact artefact that will be deployed.

Vulnerability Scan

The image scan checks:

- **OS packages**: Alpine APK packages in the base image layer, checking against NVD, Alpine SecDB, and other vulnerability databases.
- **Application dependencies**: Node.js packages installed in `node_modules`, cross-referenced against the GitHub Advisory Database and OSV.
- **Language runtime**: The Node.js binary version itself for known CVEs.

SBOM Generation

A **Software Bill of Materials** is a machine-readable inventory of every component present in a software artefact, the container equivalent of an ingredients list. Trivy generates an SBOM in **CycloneDX JSON** format for every image pushed to GHCR.

The SBOM generation step is configured as follows:

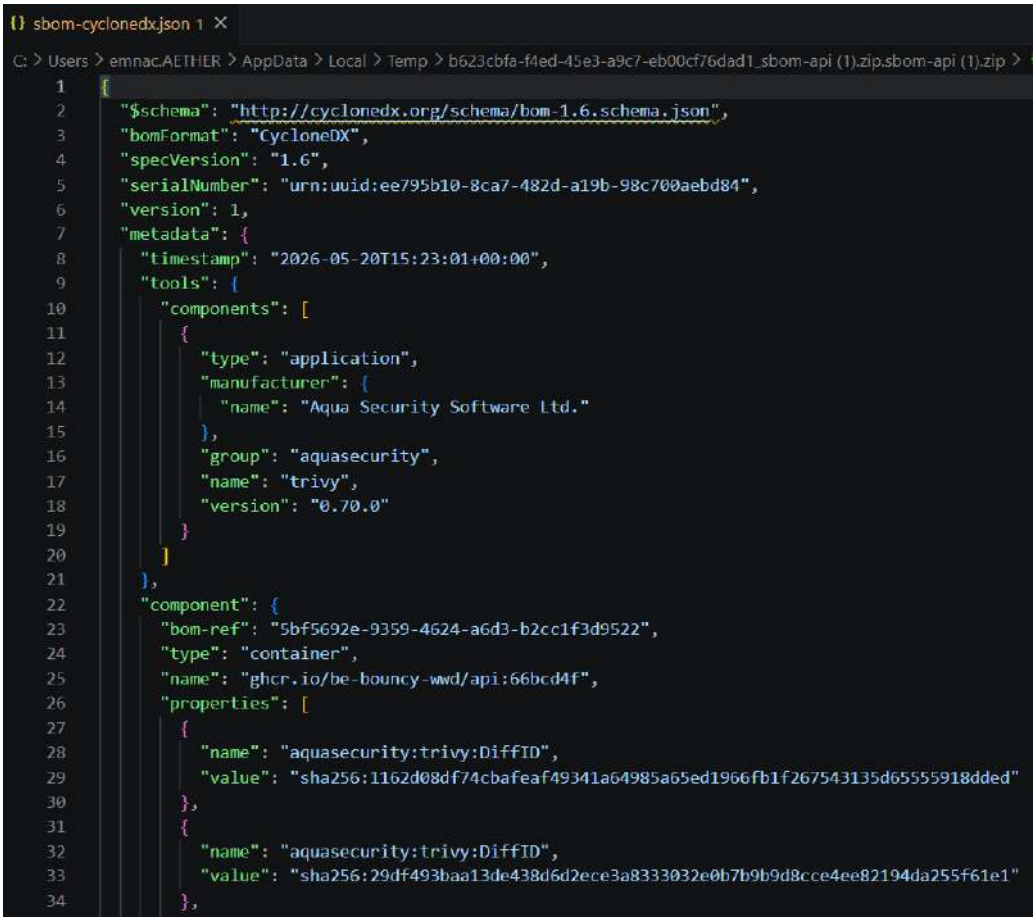


Figure 4.3: SBOM Report

The resulting SBOM file is uploaded as a GitHub Actions **pipeline artefact**, where it is retained for the configured retention period. The SBOM serves several purposes:

- **Faster vulnerability response**: The SBOM makes it possible to quickly identify whether deployed images contain newly disclosed vulnerable components.
- **Supply-chain transparency**: It provides a complete inventory of direct, transitive, and OS-level dependencies included in the container image.
- **Compliance and auditing**: SBOM generation supports security and regulatory requirements while enabling licence and dependency auditing.

4.4 Infrastructure-as-Code Security - Checkov

4.4.1 Why IaC Security Matters

The introduction of Helm charts in BB-INFRA shifted a significant amount of operational configuration into version-controlled code. This is a security improvement over manual `kubectl` commands, but it introduces a new risk: misconfigurations in Helm templates or Kubernetes manifests can silently create insecure deployments, containers running as root, missing resource limits or lack of network policies.

Checkov is a static analysis tool for Infrastructure-as-Code that understands Kubernetes YAML, Helm charts, and other IaC formats. It applies a library of security and compliance rules derived from the CIS Kubernetes Benchmark, NSA/CISA Kubernetes Hardening Guidance, and other industry standards.

4.4.2 Checkov Integration in the Pipeline

Checkov runs as a dedicated step in the pipeline, operating against the rendered output of the Helm charts in BB-INFRA. Since the application pipelines update BB-INFRA in Job 3, Checkov is invoked within the BB-INFRA update job before committing the changes, acting as a validation gate on the infrastructure manifest before it reaches ArgoCD.

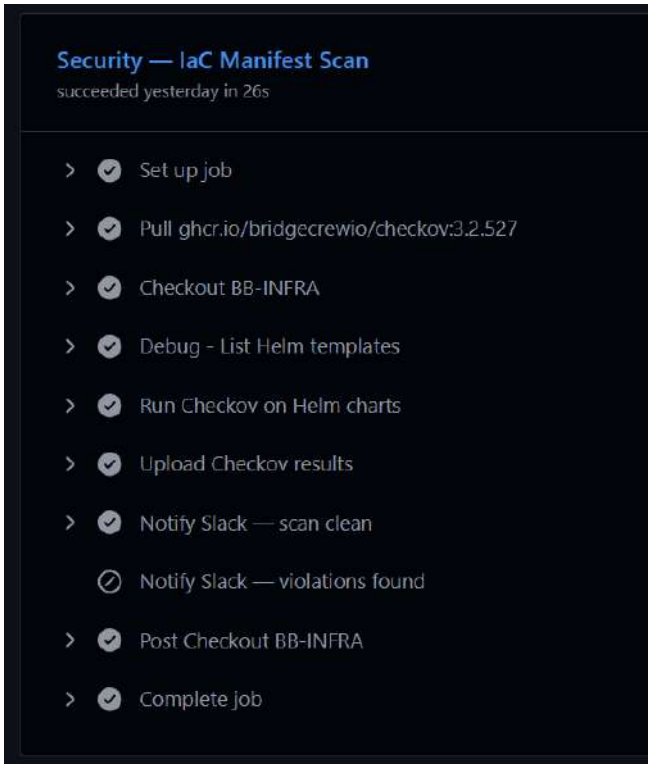


Figure 4.4: Checkov Job in the pipeline

Checkov Pipeline Artefact

Checkov generates structured scan results that are uploaded as a GitHub Actions pipeline artefact, providing a persistent and auditable record of the Kubernetes and Helm security posture for each deployment. The scan focuses on common container and orchestration security best practices, including preventing privileged or root containers, enforcing CPU and memory limits, validating readiness and liveness probes, avoiding mutable image tags such as `:latest`, and restricting dangerous Linux capabilities such as `NET_RAW`. The CLI results are also displayed directly in the pipeline logs for immediate visibility during CI execution.

4.5 Dynamic Application Security Testing - OWASP ZAP

4.5.1 What is DAST and Why it Complements SAST

Dynamic Application Security Testing interacts with a *running application* over its network interface, exactly as an attacker would. Whereas SAST analyses code structure and cannot account for runtime behaviour, DAST discovers vulnerabilities that only manifest when the application is live: server configuration exposures, missing HTTP security headers, authentication bypass at the protocol level, injection vulnerabilities that depend on database interaction, and session management flaws.

SAST and DAST are complementary, not interchangeable:

- SAST catches code-level flaws early but generates false positives and cannot test runtime behaviour.
- DAST tests real behaviour but requires a deployed environment and cannot pinpoint the source-code location of a flaw.

Together, they provide the broadest possible automated coverage of the application attack surface.

4.5.2 OWASP ZAP Integration

OWASP ZAP (Zed Attack Proxy) is the industry-standard open-source DAST tool, maintained by the Open Worldwide Application Security Project. It was integrated into the pipeline as a post-deployment Job, running against the QA environment after ArgoCD has confirmed the new version is live.

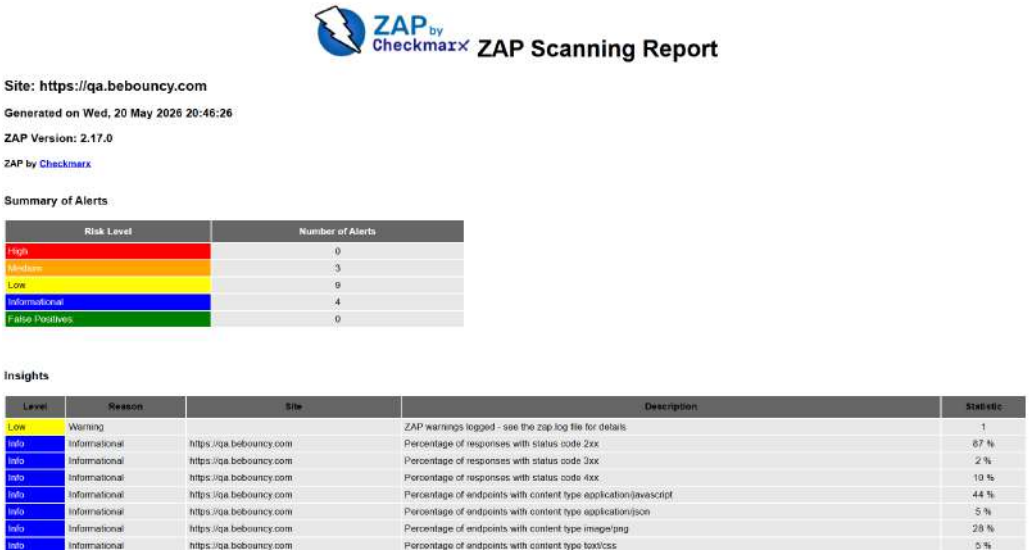
4.5.3 ZAP Pipeline Artefacts

ZAP produces two report formats, both uploaded as GitHub Actions pipeline artefacts:

Artefact	**Format**	**Purpose**
`zap-report.html`	HTML (human-readable)	Security review by engineers and stakeholders
`zap-report.json`	JSON (machine-readable)	Integration with dashboards, tracking tools

Table 4.3: OWASP ZAP Pipeline Artefacts

The HTML report groups findings by risk level (High, Medium, Low, Informational), provides a description of each finding, the URL at which it was detected, the evidence collected, and recommended remediation steps. The JSON report contains the same information in a structured format suitable for programmatic processing.



ZAP by Checkmarx ZAP Scanning Report

Site: https://qa.bebouncy.com

Generated on Wed, 20 May 2026 20:46:26

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	3
Low	9
Informational	4
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	1
Info	Informational	https://qa.bebouncy.com	Percentage of responses with status code 2xx	87 %
Info	Informational	https://qa.bebouncy.com	Percentage of responses with status code 3xx	2 %
Info	Informational	https://qa.bebouncy.com	Percentage of responses with status code 4xx	10 %
Info	Informational	https://qa.bebouncy.com	Percentage of endpoints with content type application/javascript	44 %
Info	Informational	https://qa.bebouncy.com	Percentage of endpoints with content type application/json	5 %
Info	Informational	https://qa.bebouncy.com	Percentage of endpoints with content type image/png	28 %
Info	Informational	https://qa.bebouncy.com	Percentage of endpoints with content type text/css	5 %

Figure 4.5: Owasp Zap HTML Report

4.6 Secret Management

4.6.1 The Problem with `.env` Files

Prior to this release, application secrets such as database connection strings, API keys for OpenAI, Gemini, Facebook Graph API, LinkedIn, and OVH were distributed to the QA server as `.env` files. This approach has several critical weaknesses:

- Files on disk can be read by any process running with the same user privileges.
- Secrets are static; rotating a credential requires manually updating the file and restarting the service.
- If a server is compromised, all secrets are immediately available to the attacker in plaintext.
- There is no audit log of which process read which secret, or when.
- Human error can cause secrets to be committed to version control alongside the `.env` file.

4.6.2 HashiCorp Vault as the Central Secret Store

HashiCorp Vault is an identity-based secrets management platform. All application secrets for the BeBouncy platform are stored in Vault, which was already partially integrated before this release. Release 2 completes that integration, ensuring that *no application secret ever appears outside of Vault* except transiently at runtime within the memory of the process that needs it.

Vault Architecture for BeBouncy

Vault is accessed by the CI/CD pipeline and by Kubernetes pods via authenticated API calls. The key design decisions are:

- **Secret paths**: Secrets are organised under environment-namespaced paths, e.g., `secret/qa/api`, ensuring environment isolation within a single Vault instance.
- **Authentication**: The pipeline authenticates to Vault using the GitHub Actions OIDC token (short-lived, no static Vault token stored in GitHub Secrets). Kubernetes pods authenticate using the Kubernetes auth method, which validates the pod's service account token against the Kubernetes API.
- **Policies**: A least-privilege Vault policy is defined per service, granting `read` access only to the specific secret path that service requires. The API service cannot read AI service secrets, and neither can read infrastructure secrets.
- **Lease TTLs**: Dynamic credentials are issued with short TTLs and automatically renewed by the application, eliminating long-lived static credentials entirely.

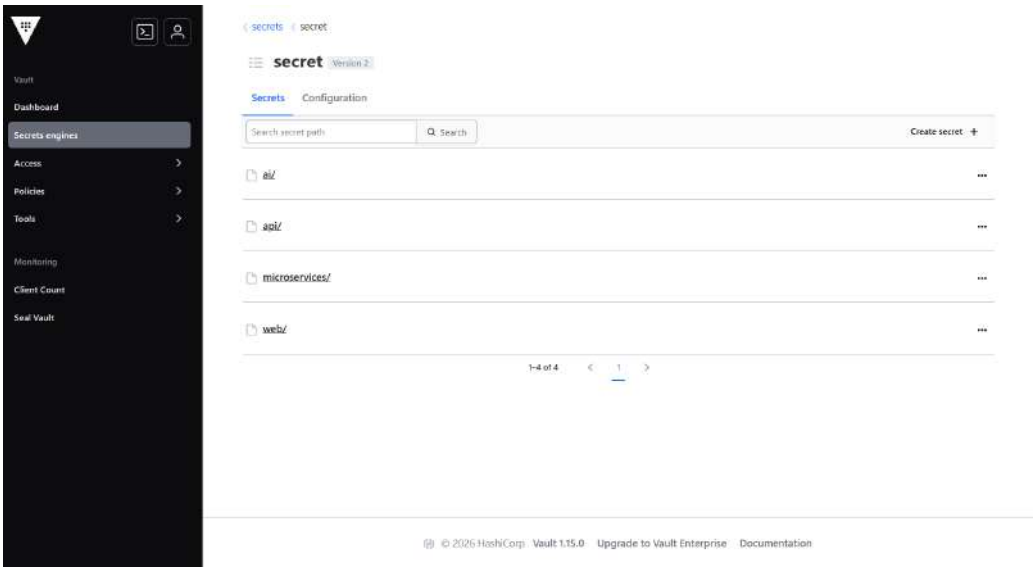


Figure 4.6: HashiCorp Vault Overview

4.6.3 Kubernetes Secrets as the Runtime Injection Mechanism

Kubernetes Secrets are the mechanism by which those secrets are delivered to running pods. The integration works as follows:

1. The Helm chart for each service references these Kubernetes Secrets via `envFrom` or `env.valueFrom.secretKey`
   in the pod specification.

2. When Kubernetes schedules a pod, it injects the secret values as environment variables into the container's runtime environment, they are never written to disk and never appear in the container image or in the Helm chart YAML committed to BB-INFRA.

Security Properties of This Architecture

- **No secrets in Git**: Neither BB-INFRA nor any application repository contains a secret value at any commit in its history (after remediation — see Gitleaks section below).
- **No secrets in images**: Container images contain no secret material; they are safe to share and inspect.
- **No secrets on disk**: Secrets exist only in Vault's encrypted storage, in Kubernetes' etcd store (encrypted at rest), and transiently in pod memory.
- **Audit logging**: Vault logs every read request, providing a complete audit trail of which entity accessed which secret and when.
- **Secret rotation**: Rotating a credential requires updating the value in Vault and triggering a pod restart (rolling update); the pipeline and application code require no changes.

4.7 Alignment with Security Compliance Frameworks

As a SaaS platform handling user data, BeBouncy operates in a context where security compliance is a business requirement as much as a technical concern. The DevSecOps controls implemented in this release support alignment with three widely recognised frameworks: **ISO/IEC 27001**, **SOC 2**, and the **GDPR**.

ISO/IEC 27001 defines requirements for establishing an Information Security Management System (ISMS) and managing risks through structured controls. The implemented pipeline maps directly to several Annex A domains: vulnerability management is enforced through SonarCloud static analysis and Trivy container scanning; secure development practices are supported via Checkov Infrastructure-as-Code validation and CI/CD quality gates; and access control is strengthened through HashiCorp Vault and Kubernetes Secrets, ensuring least-privilege handling of sensitive data.

SOC 2 is an auditing framework commonly required for SaaS providers, focusing on security, availability, confidentiality, processing integrity, and privacy. The pipeline generates continuous, traceable evidence through Trivy SBOM reports, OWASP ZAP security testing outputs, and Checkov scan results. These artefacts support auditability by demonstrating that security controls are consistently applied rather than verified only periodically.

GDPR requires appropriate technical safeguards for personal data protection. In this implementation, Vault-based secret management ensures that credentials are never stored in plaintext and can be rotated securely with full auditability. In addition, OWASP ZAP dynamic testing helps detect runtime vulnerabilities such as insecure headers or session weaknesses that could expose user data.

Overall, while these controls do not constitute a full certification-ready compliance programme, they provide a strong technical foundation. The automated, evidence-driven pipeline nevertheless significantly reduces the effort required to achieve such compliance milestones.

4.8 Complete Security-Enhanced Pipeline Flow

4.8.1 Revised Pipeline Architecture

The full pipeline after Release 2 integrates all security controls into the three-job structure established in Release 1, with one additional post-deploy job for DAST

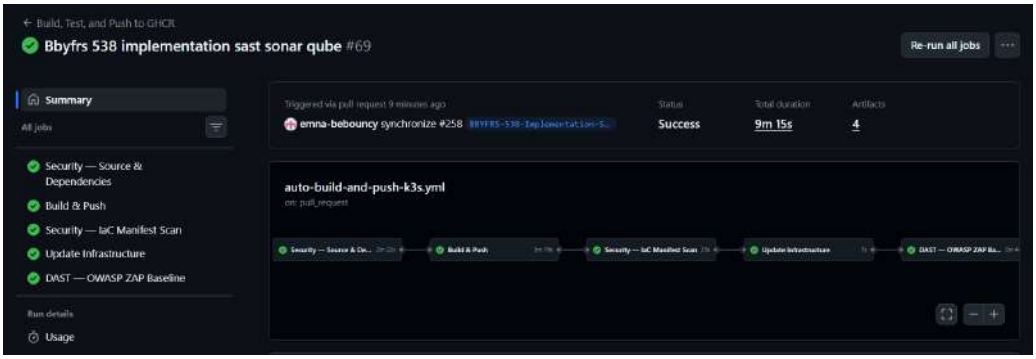


Figure 4.7: Full Pipeline

Figure 4.7 illustrates the pipeline artefacts generated and uploaded during runtime by the different stages of the DevSecOps pipeline, providing traceability, auditability, and persistent records of each pipeline execution.

Figure 4.8: Produced Artifacts

Slack notifications are integrated throughout the pipeline to provide immediate feedback on the

status of each stage. A notification is sent automatically after every successful or failed job, allowing deployment progress, security scan outcomes, and infrastructure updates to be monitored in real time without requiring direct access to GitHub Actions logs.

Figure 4.9 shows an example of a successful pipeline notification sent after the completion of the build and deployment workflow.

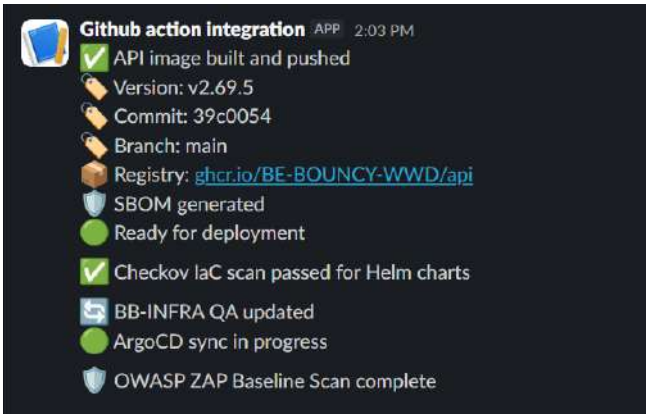


Figure 4.9: Slack Notification of a successful pipeline

Conclusion

Release 2 elevates the BeBouncy DevSecOps pipeline from a functional delivery mechanism to a hardened, security-integrated engineering system. By embedding SonarCloud SAST, Trivy filesystem and image scanning with SBOM generation, Checkov IaC validation, OWASP ZAP dynamic testing, and a comprehensive secret management architecture into the automated pipeline, security becomes integrated into every stage of the software lifecycle, from the first commit to the running pod.

The implemented defence-in-depth model ensures that multiple layers of security controls work together to reduce risk. Each tool addresses a different attack surface: source-code vulnerabilities, insecure dependencies, container image risks, infrastructure misconfigurations, runtime exposure, and secret leakage. Beyond technical hardening, the automated and evidence-producing nature of the pipeline lays the groundwork for future alignment with security compliance frameworks such as ISO 27001, SOC 2, and GDPR, reducing the engineering effort required to reach those milestones. The result is a pipeline capable of validating artefacts before deployment, dynamically testing running applications, and securely handling sensitive credentials through HashiCorp Vault and Kubernetes Secrets.

With the delivery pipeline now both automated and hardened, the project proceeds to its third and final release: operational observability, covering Prometheus metrics collection, Grafana dashboarding, and AlertManager-based notification, providing the visibility required to operate the platform confidently in a production-like environment.

Chapter 5

Sprint Release 3: Monitoring and Observability

Contents

Introduction

The first two releases of the BeBouncy DevSecOps project established an automated, security-focused delivery pipeline and a GitOps-driven Kubernetes environment. However, deploying applications reliably is only part of operating a platform effectively; continuous visibility into system health and performance is equally essential.

To address this, Release 3 introduces a complete monitoring and observability stack integrated into the existing GitOps workflow. This enables proactive detection of infrastructure and application issues, helping ensure the platform can be monitored and operated reliably in a production-like environment.

1. **Metrics collection** via Prometheus, scraping the Kubernetes cluster, node hardware, and all running workloads at a configurable interval.
2. **Visual dashboarding** via Grafana, presenting cluster-wide and per-service resource consumption through a purpose-built custom dashboard alongside the industry-standard Node Exporter Full dashboard.
3. **Proactive alerting** via AlertManager, evaluating Prometheus rules against the collected metrics and dispatching email notifications to the engineering team's work inbox when resource thresholds are breached or workload health degrades and again when conditions resolve.

The entire stack is deployed as a single Helm-managed ArgoCD application backed by the `kube-prometheus-stack` chart from the Prometheus Community repository, with a companion custom Helm chart in BB-INFRA that manages the alerting rules as a Kubernetes-native `PrometheusRule` resource.

5.1 Observability Architecture

5.1.1 Design Philosophy

The observability stack was designed around three principles that mirror the broader project philosophy:

GitOps-native deployment The monitoring stack is not installed manually on the QA node. Like every other component in the cluster, it is declared in BB-INFRA and reconciled by ArgoCD. Adding the two manifest files: `monitoring-app.yaml` and `monitoring-rules-app.yaml` and pushing them to the `main` branch was sufficient to cause ArgoCD to create the `monitoring` namespace and deploy all six monitoring pods without any SSH session or manual `kubectl` command.

Separation of concerns The Prometheus alerting rules are managed in a *separate* ArgoCD application (`monitoring-rules-qa`) backed by a dedicated Helm chart in `charts/monitoring/`. This separation means alerting rules can be updated independently of the monitoring stack itself, a rule change does not trigger a re-deployment of Prometheus or Grafana.

5.1.2 Component Overview

The monitoring stack in the `monitoring` namespace consists of the following pods, as observed on the running cluster:

```
ubuntu@b2-7-rbx-a:~$ kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS      AGE
alertmanager-monitoring-qa-kube-prometh-alertmanager-0   2/2     Running   0             4h6m
monitoring-qa-grafana-6d4b964574-tk7v6                   3/3     Running   3 (10h ago)   2d2h
monitoring-qa-kube-prometh-operator-54665b9484-pxb7s     1/1     Running   1 (10h ago)   2d9h
monitoring-qa-kube-state-metrics-544d75dd56-bzkdc        1/1     Running   1 (10h ago)   2d9h
monitoring-qa-prometheus-node-exporter-vh4p8             1/1     Running   1 (10h ago)   2d9h
prometheus-monitoring-qa-kube-prometh-prometheus-0       2/2     Running   3 (8h ago)    2d9h
ubuntu@b2-7-rbx-a:~$ kubectl get deployments -n monitoring
NAME                                  READY   UP-TO-DATE   AVAILABLE   AGE
monitoring-qa-grafana                 1/1     1            1           2d9h
monitoring-qa-kube-prometh-operator   1/1     1            1           2d9h
monitoring-qa-kube-state-metrics      1/1     1            1           2d9h
ubuntu@b2-7-rbx-a:~$ kubectl get svc -n monitoring
NAME                                     TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
alertmanager-operated                    ClusterIP   None            <none>        9093/TCP,9094/TCP,9094/UDP   4h7m
monitoring-qa-grafana                    ClusterIP   10.43.110.66    <none>        80/TCP                       2d9h
monitoring-qa-kube-prometh-alertmanager  ClusterIP   10.43.199.156   <none>        9093/TCP,8080/TCP            6h13m
monitoring-qa-kube-prometh-operator      ClusterIP   10.43.143.38    <none>        8080/TCP                     2d9h
monitoring-qa-kube-prometh-prometheus    ClusterIP   10.43.219.123   <none>        9090/TCP,8080/TCP            2d9h
monitoring-qa-kube-state-metrics         ClusterIP   10.43.214.6     <none>        8080/TCP                     2d9h
monitoring-qa-prometheus-node-exporter   ClusterIP   10.43.106.145   <none>        9100/TCP                     2d9h
prometheus-operated                      ClusterIP   None            <none>        9090/TCP                     2d9h
```

Figure 5.1: State of Pods, deployments and service of Monitoring namespace

5.1.3 Data Flow

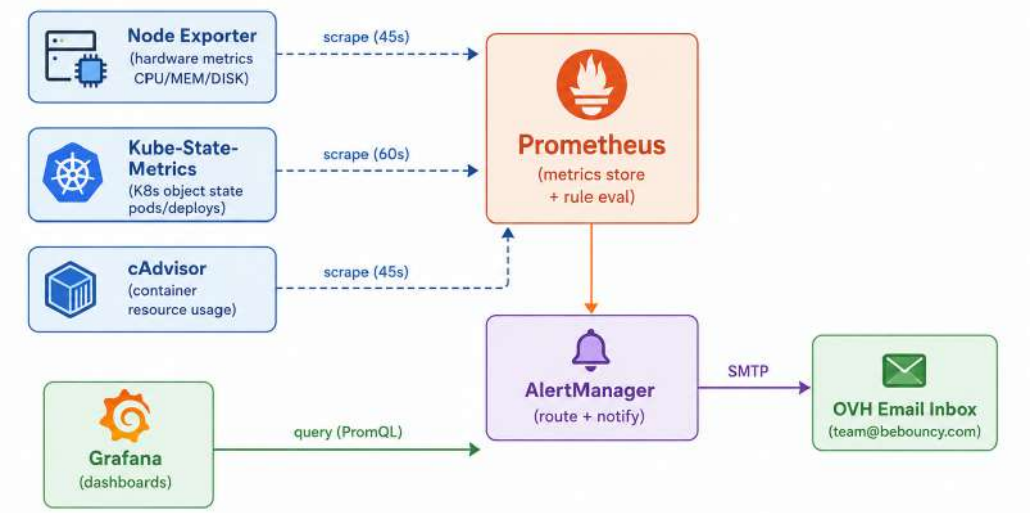


Figure 5.2: Observability Data Flow

Figure 5.2 illustrates the observability data flow implemented in the QA Kubernetes cluster. Infrastructure, Kubernetes, and container-level metrics are collected by Node Exporter, Kube-State-Metrics, and cAdvisor, then scraped and stored by Prometheus for querying and rule evaluation. Grafana retrieves these metrics using PromQL to provide real-time dashboards, while AlertManager processes alert rules and sends email notifications through the OVH SMTP service when predefined thresholds are exceeded.

5.2 Deployment via GitOps

5.2.1 The Two ArgoCD Applications

The monitoring stack is managed through two ArgoCD `Application` resources committed to BB-INFRA under `argocd/qa/`:

`monitoring-app.yaml` Deploys the full `kube-prometheus-stack` Helm chart from the official Prometheus Community Helm repository (chart version `61.7.2`). This application uses a *multi-source* ArgoCD configuration: the chart is sourced from the upstream Helm repository, while the values file is sourced from BB-INFRA itself. This pattern allows the upstream chart to be version-pinned and updated independently of the configuration values, while the values remain version-controlled in Git.

`monitoring-rules-app.yaml` Deploys a custom Helm chart which contains a single template: a `PrometheusRule` custom resource that defines all alerting rules for the QA cluster. By separating rules into their own ArgoCD application, a rule change (such as adjusting a threshold from 85% to 80%) triggers only a re-application of the `PrometheusRule` object, not a rolling restart of the Prometheus server.

5.2.2 Multi-Source ArgoCD Application Pattern

The `monitoring-app.yaml` manifest demonstrates an advanced ArgoCD feature, the multi-source application which enables referencing a Helm chart from one repository while loading its values from another:

monitoring-app.yaml

argocd > qa > monitoring-app.yaml

```yaml
spec:
  project: default
  sources:
    # Source 1: the Helm chart from prometheus-community
    - repoURL: https://prometheus-community.github.io/helm-charts
      chart: kube-prometheus-stack
      targetRevision: "61.7.2"
      helm:
        valueFiles:
          - $values/BB-INFRA/environments/qa/values-monitoring.yaml
    # Source 2: BB-INFRA values
    - repoURL: https://github.com/BE-BOUNCY-WWD/BB-INFRA.git
      targetRevision: main
      ref: values
```

Figure 5.3: Multi-source Application

The second source is assigned the alias `$values`; it does not contribute any Kubernetes resources directly but makes the BB-INFRA repository available as a path reference for the `valueFiles` field of the first source. This is the canonical pattern for consuming upstream community charts while maintaining environment-specific configuration in a private GitOps repository.

5.3 Prometheus - Metrics Collection and Storage

5.3.1 What is Prometheus?

Prometheus is an open-source systems monitoring and alerting toolkit, part of the Cloud Native Computing Foundation (CNCF). It operates on a *pull model*: rather than having applications push metrics to a central server, Prometheus scrapes HTTP endpoints exposed by targets at a configured interval, storing the resulting time-series data in its local time-series database (TSDB). Queries against this database use **PromQL** (Prometheus Query Language), a functional expression language that powers both dashboards and alerting rules.

5.3.2 Prometheus Operator and the kube-prometheus-stack

The `kube-prometheus-stack` chart deploys Prometheus via the **Prometheus Operator** a Kubernetes controller that manages Prometheus instances and their configuration through custom resource definitions (CRDs). Instead of editing a static configuration file, the operator watches for `ServiceMonitor`, `PodMonitor`, and `PrometheusRule` objects and automatically updates the Prometheus configuration without requiring a pod restart. This is the standard production pattern for running Prometheus on Kubernetes.

5.3.3 Prometheus Configuration for the QA Cluster

The Prometheus instance is configured in `values-monitoring.yaml` and deployed in the QA cluster is configured for a lightweight, single-node environment with one replica and persistent storage backed by an 8 Gi local-path PVC. Metrics are retained for three days with a maximum storage limit of 7 GB to control TSDB growth. Both the scrape and rule evaluation intervals are set to 45 seconds, providing sufficient monitoring granularity for the QA workload while reducing unnecessary resource consumption. CPU and memory requests/limits are also right-sized to balance stability and efficient resource usage on the cluster.

Persistent Storage

Prometheus stores collected metrics on a persistent 8 GiB volume so that monitoring data is preserved even if the pod restarts or the node reboots. To avoid excessive disk usage on the QA server, metric retention is limited to three days with a maximum storage size of 7 GB.

Collected Metrics

The monitoring stack collects metrics from several sources within the Kubernetes cluster. Node Exporter provides infrastructure metrics such as CPU, memory, disk, and network usage, while Kube-State-Metrics exposes Kubernetes object information including pod states, deployment replicas, and restart counts. Container-level resource usage is collected through cAdvisor, and additional metrics are gathered from CoreDNS, the Kubernetes API server, and the kubelet for cluster health monitoring.

Some control-plane components such as `etcd`, the scheduler, and the controller manager were not monitored because they are not exposed separately in this single-node Kubernetes distribution.

Automatic Service Discovery

Prometheus is configured to automatically discover monitoring targets across all namespaces. This means future applications can expose their own custom metrics simply by adding a `ServiceMonitor` resource, without requiring manual changes to the Prometheus configuration.

5.4 Grafana - Dashboarding and Visualisation

5.4.1 What is Grafana?

Grafana is an open-source analytics and interactive visualisation platform. It connects to Prometheus as a data source, executes PromQL queries, and renders the results as time-series graphs, stat panels, bar charts, and tables, organised into composable dashboards. Dashboards are the primary interface through which engineers observe the health and resource consumption of the cluster in real time.

5.4.2 Accessing the Grafana UI

For security reasons, Grafana is not exposed publicly through an Ingress resource. Instead, access is restricted to authenticated developers through a secure combination of Kubernetes port-forwarding and SSH tunnelling between the QA server and the local workstation.

This approach allows the Grafana dashboard to remain isolated from the public internet while still providing secure remote access for monitoring and operational tasks. The same access pattern is also used for the AlertManager interface within the monitoring stack.

5.4.3 Dashboards

Two dashboards are provisioned automatically via the `dashboardProviders` and `dashboards` configuration in `values-monitoring.yaml`. Both are placed in the **"QA Cluster"** folder within Grafana and are available immediately after ArgoCD syncs the application, no manual import is required.

5.4.3.1 Node Exporter Full Dashboard

The **Node Exporter Full** dashboard is imported directly from the Grafana dashboard registry. It provides a comprehensive view of the underlying host machine's resource consumption.

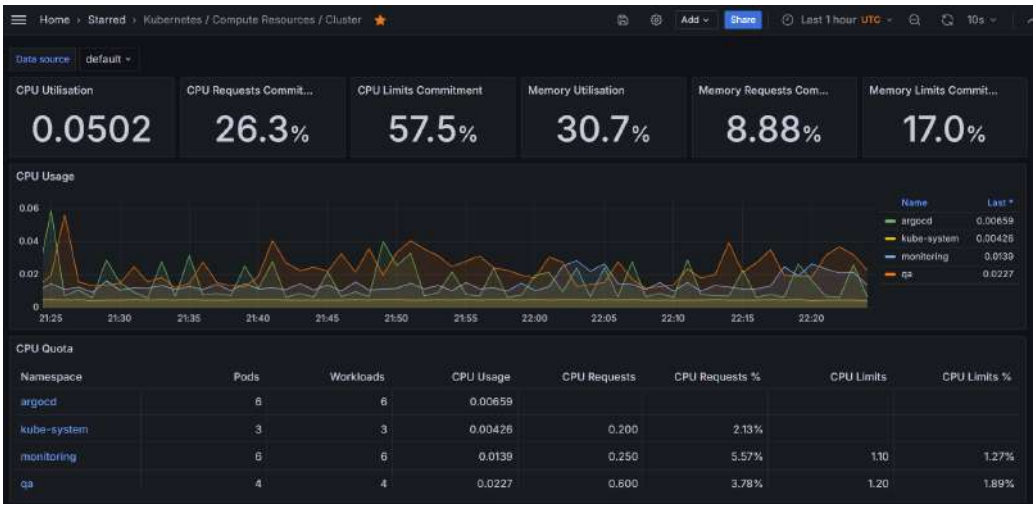


Figure 5.4: Overview of Host machine's resources consumption

This dashboard is the standard reference for diagnosing node-level performance issues and understanding the hardware utilisation baseline of the QA server.

5.4.3.2 QA Cluster Custom Dashboard

The second dashboard is a purpose-built custom dashboard authored specifically for the BeBouncy QA cluster. It is defined entirely as a JSON payload within `values-monitoring.yaml`, making it version-controlled in BB-INFRA alongside all other infrastructure configuration. The dashboard is organised into five sections, each targeting a distinct observability concern:

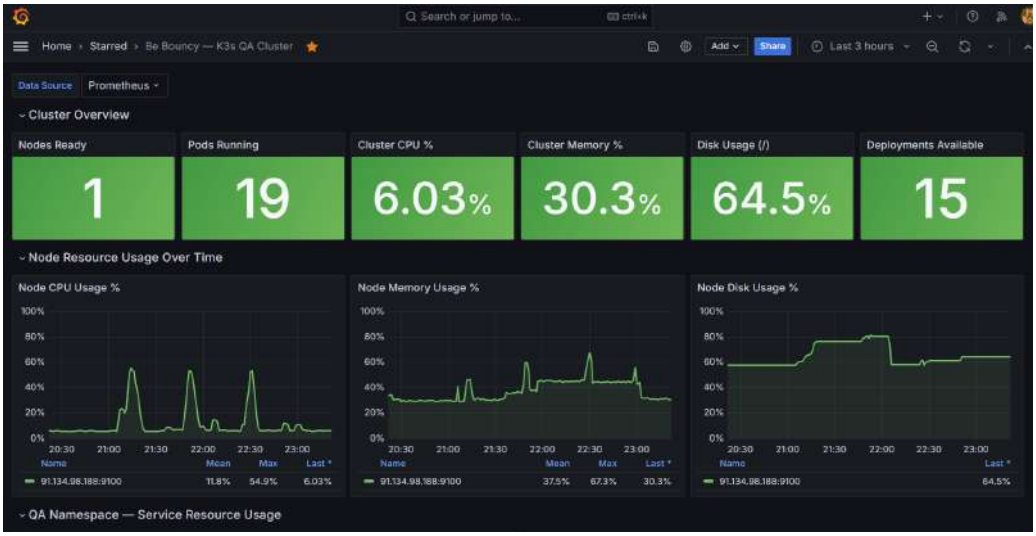


Figure 5.5: QA Cluster Grafana Dashboard

- **Cluster Overview**: Colour-coded status panels provide a quick summary of cluster health, including node readiness, running pods, CPU and memory utilisation, disk usage, and deployment availability.
- **Node Resource Monitoring**: Time-series graphs display CPU, memory, and disk usage over time, helping identify performance trends and abnormal resource spikes.
- **QA Namespace Monitoring**: Dedicated panels track CPU and memory consumption for application pods in the `qa` namespace, making it easier to identify resource-intensive services.
- **Namespace Comparison**: Cluster-wide resource usage is grouped by namespace to compare the consumption of application, monitoring, and system components.
- **Workload Health**: Table panels display deployment replica status and pod restart counts, helping detect unhealthy or unstable workloads within the cluster.

5.5 AlertManager - Proactive Alerting

5.5.1 What is AlertManager?

AlertManager is the alerting component of the Prometheus ecosystem. Prometheus evaluates *alerting rules* against its collected metrics at each evaluation interval and sends *firing* alerts to AlertManager when a rule's threshold is breached. AlertManager is then responsible for *routing*, *grouping*, *silencing*, *inhibiting*, and *dispatching* those alerts to the appropriate receivers in this case, an email address via SMTP.

AlertManager decouples the act of *detecting* a problem (Prometheus) from the act of *notifying*

someone about it (AlertManager). This separation allows routing logic, notification throttling, and receiver configuration to evolve independently of the alerting rules themselves.

5.5.2 AlertManager Configuration

AlertManager is configured through the `alertmanager.config` section of `values-monitoring.yaml`. The key configuration decisions are:

SMTP Transport

AlertManager sends email notifications through the OVH SMTP relay using the `emna@bebouncy.com` account. For security, the SMTP password is stored as a Kubernetes `Secret` in the `monitoring` namespace and mounted into the AlertManager pod at runtime, ensuring sensitive credentials are never exposed in Helm values files, Git repositories, or environment variables.

Routing Logic

AlertManager groups related alerts together before sending notifications in order to reduce alert noise and avoid excessive emails. A short delay is applied before the first notification to allow similar alerts to be grouped, while repeated notifications for ongoing issues are limited to periodic intervals. All alerts are routed to the default `email-alerts` receiver configured for the monitoring stack.

Watchdog Suppression

The `kube-prometheus-stack` chart includes a built-in **Watchdog** alert that fires continuously at all times as a heartbeat, it confirms that the Prometheus and AlertManager pipeline is functioning end-to-end. Without a specific route, this alert would generate an email every 12 hours. A dedicated route matches the `Watchdog` alertname and directs it to the `null` receiver (which discards the notification), keeping the inbox clean while preserving the heartbeat function.

Inhibition Rules

AlertManager is configured to suppress lower-severity alerts when a related critical alert is already active. This reduces duplicate notifications and prevents alert flooding caused by the same underlying issue.

Email Notification Format

Alert emails use a custom subject format that clearly indicates the alert status and affected issue, allowing problems to be identified quickly without opening the email. Notifications are also sent automatically when an alert is resolved, ensuring the full alert lifecycle is communicated to the

operations team.

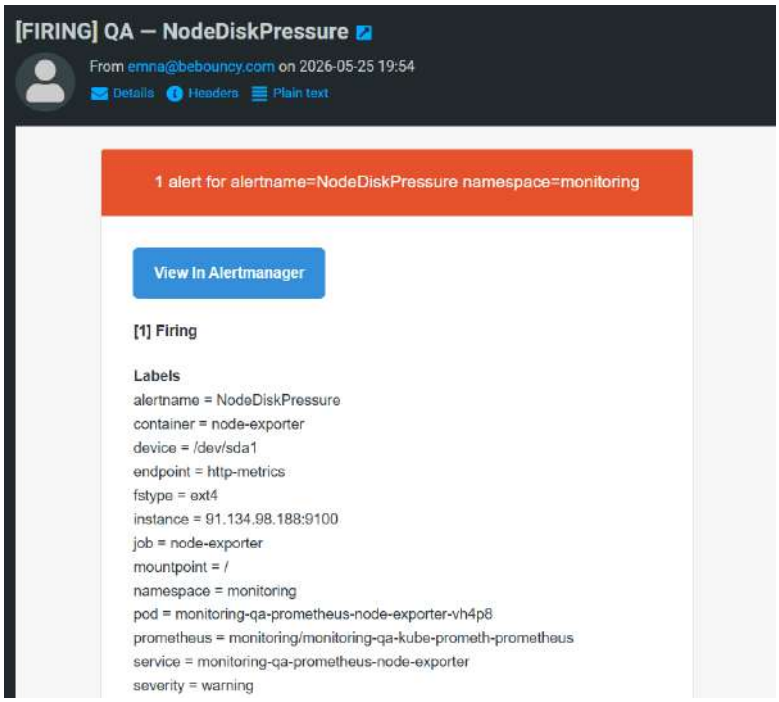


Figure 5.6: Email alert firing

Figures 5.6 shows an example of the alerting workflow for the `NodeDiskPressure` alert. The first notification was triggered when root disk utilisation exceeded the configured threshold, generating a warning email containing the alert name, affected node, severity level, and diagnostic metadata collected by Prometheus and AlertManager.

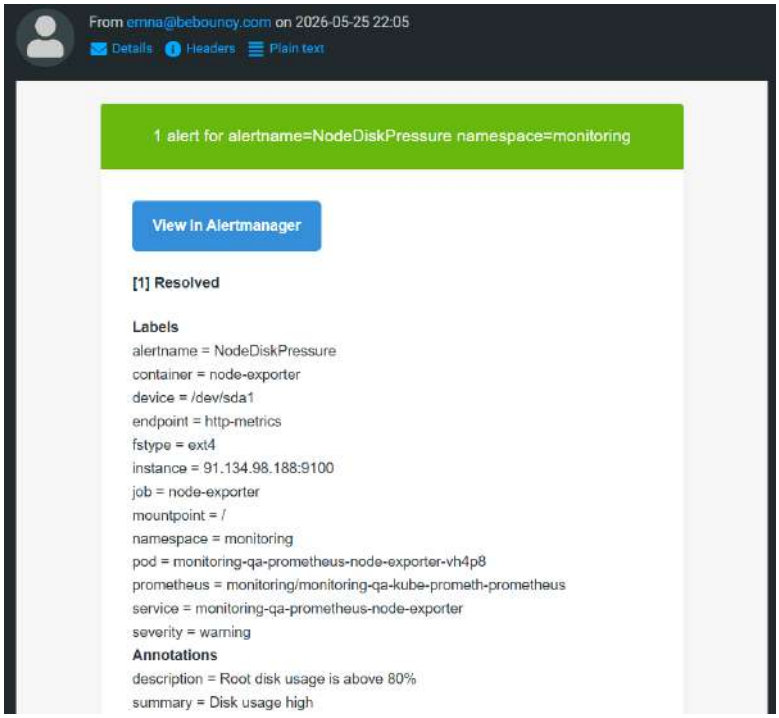


Figure 5.7: Email alert resolved

Once disk utilisation returned to a healthy level, a second notification was automatically sent indicating that the alert had been resolved. This confirms that the monitoring stack not only detects operational issues proactively, but also communicates their resolution without requiring manual verification through the AlertManager interface.

5.6 Prometheus Alerting Rules

5.6.1 PrometheusRule Custom Resource

Alerting rules are defined in `prometheus-rules.yaml` as a `PrometheusRule` custom resource. The Prometheus Operator watches for this resource type in the `monitoring` namespace and automatically loads the rules into the running Prometheus instance. Updating a rule requires only a Git commit; the `monitoring-rules-qa` ArgoCD application syncs the change within minutes, and the Prometheus Operator applies the updated `PrometheusRule` without restarting the Prometheus pod.

5.6.2 Rule Groups

The rules are organised into two groups, each with a 60-second evaluation interval:

Group 1: `node.alerts` **- Infrastructure Health**

Alert Name	Condition	For	Severity
`NodeHighCPU`	CPU usage > 85%	3 minutes	Warning
`NodeHighMemory`	Memory usage > 85%	3 minutes	Warning
`NodeDiskPressure`	Root filesystem usage > 80%	2 minutes	Warning

Table 5.1: Node Infrastructure Alert Rules

The `for` duration prevents transient spikes from generating alerts. A CPU burst that lasts 30 seconds during a deployment will not trigger `NodeHighCPU`; the condition must be sustained for 3 continuous minutes before AlertManager dispatches a notification. This significantly reduces alert fatigue in a QA environment where deployments regularly cause brief resource spikes.

Group 2: `kubernetes.alerts` **- Workload Health**

The `PodCrashLooping` rule computes the rate of restarts over the last 15 minutes rather than comparing an absolute restart count. This makes the alert sensitive to new crash loops while remaining immune to pods that have a high historical restart count but have been stable for an extended period.

Alert Name	Condition	For	Severity
`PodCrashLooping`	Pod restart rate > 0 over 15 minutes	2 minutes	Critical
`PodNotReady`	Any pod in Pending / Unknown / Failed phase	2 minutes	Warning
`DeploymentReplicasMismatch`	Available replicas < desired replicas	3 minutes	Critical

Table 5.2: Kubernetes Workload Alert Rules

5.7 SRE Concepts: SLI, SLO, and Error Budget

5.7.1 From Alerting to Service Reliability

The alerting rules defined in the previous section ensure that the team is notified when infrastructure thresholds are breached. However, operational excellence in modern platform engineering goes one step further, moving from *reactive* threshold monitoring toward *quantified* reliability targets. This is the domain of **Site Reliability Engineering (SRE)**, a discipline introduced by Google that provides a structured framework for defining, measuring, and maintaining service reliability.

5.7.2 Key Concepts

SRE introduces three complementary concepts that form the foundation of reliability measurement:

Service Level Indicator (SLI) A quantitative measure of a specific aspect of service behaviour, expressed as a ratio or rate computed from raw metrics. Typical SLIs include request success rate, latency below a given threshold, and uptime. For BeBouncy, a representative SLI would be the proportion of HTTP requests completed successfully (HTTP 2xx or 3xx) relative to the total number of requests received.

Service Level Objective (SLO) A target value or range set for an SLI over a defined time window. An SLO translates an abstract reliability goal into a concrete, measurable commitment. For example: *"99% of HTTP requests to the QA environment must succeed over any rolling 30-day window."* The SLO is an internal engineering target, not a contractual guarantee.

Service Level Agreement (SLA) A formal contract between a service provider and its customers, specifying the minimum acceptable level of service and the consequences of failing to meet it. SLAs are typically less strict than SLOs, since SLOs include a safety margin to avoid breaching the SLA.

5.7.3 Error Budget

The **error budget** is derived directly from the SLO: it represents the permissible amount of unreliability within the measurement window. For an SLO of 99% availability over 30 days, the error budget is 1%, which corresponds to approximately 432 minutes of allowable downtime per month. As long as the error budget has not been exhausted, the team retains the freedom to release new features and accept the associated deployment risk. Once the budget is consumed, reliability work takes priority over feature delivery. This mechanism aligns development velocity with operational stability in a measurable, objective way.

5.7.4 Relevance to the BeBouncy Monitoring Stack

The Prometheus and Grafana stack deployed in this release provides the technical foundation required to implement SLO tracking in the future. Prometheus already collects the raw metrics necessary to compute availability-related SLIs. For instance, the fraction of time during which all QA namespace pods are in a `Ready` state can be expressed as a PromQL recording rule, and the cumulative burn of the error budget can be visualised as a Grafana time-series panel. The `DeploymentReplicasMismatch` and `PodNotReady` alerting rules defined in Section 5.6 serve as early SLO-burn indicators: a firing `DeploymentReplicasMismatch` alert signals that the service is actively consuming its error budget and immediate action is required to protect the SLO. Formalising these measurements into explicit SLOs and error budget dashboards represents a natural next step as the platform matures toward a production-grade environment.

5.8 Complete Observability Workflow

During normal operation, Prometheus continuously scrapes metrics from cluster components every 45 seconds, while Grafana queries this data using PromQL to display it in dashboards. Engineers access these dashboards via a port-forward tunnel to monitor cluster health, including CPU, memory, disk usage, resource trends, pod-level consumption, deployment status, and any non-zero pod restarts.

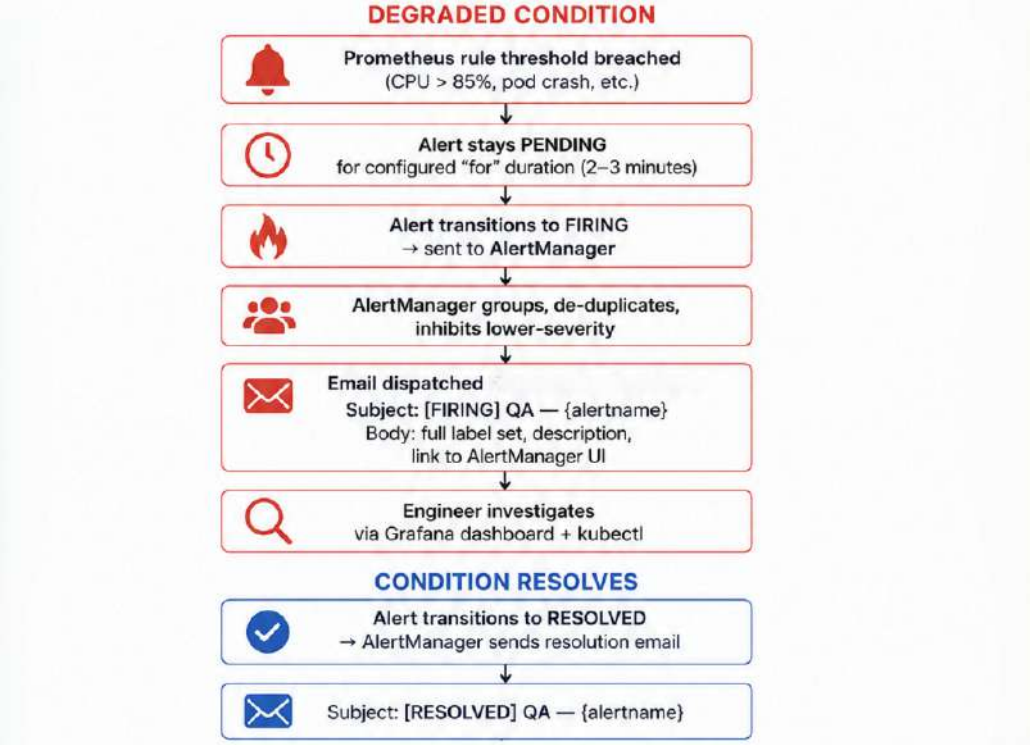


Figure 5.8: Observability workflow

Conclusion

Release 3 closes the observability gap that remained after the first two releases, completing the BeBouncy DevSecOps platform. The QA cluster is now fully instrumented through three integrated components: Prometheus continuously scrapes metrics from the node, Kubernetes control plane, and every running container; Grafana surfaces those metrics through a version-controlled custom dashboard covering cluster health, per-namespace consumption, and workload stability; and AlertManager dispatches email notifications when thresholds are breached and again when conditions resolve, covering the full alert lifecycle without manual intervention. Beyond reactive alerting, the concepts of SLI, SLO, and error budget introduced in this chapter establish the theoretical groundwork for evolving the platform toward quantified, SRE-grade reliability measurement as BeBouncy moves closer to a production environment.

Conclusion

This internship project provided a valuable opportunity to apply and strengthen the theoretical knowledge acquired during the academic program within a real-world DevSecOps environment. It supported the progressive design and implementation of a modern infrastructure for BeBouncy, organized into three iterative releases that gradually improved reliability, security, and observability.

The **first release** focused on eliminating environment inconsistencies and manual deployment processes. This was achieved by containerizing the application's four services using optimized multi-stage Dockerfiles, deploying them on a single-node Kubernetes cluster hosted on the QA server, and introducing a GitOps workflow using Helm, BB-INFRA, and ArgoCD. A three-stage GitHub Actions pipeline was also implemented to fully automate the delivery process from code commit to deployment, removing any need for manual server intervention.

The **second release** strengthened the platform by integrating security across the delivery pipeline. Multiple controls were introduced, including SonarCloud for static code analysis, Trivy for container and filesystem vulnerability scanning with SBOM generation, Checkov for Infrastructure-as-Code validation, OWASP ZAP for dynamic security testing, and HashiCorp Vault combined with Kubernetes Secrets for secure secret management at runtime.

The **third release** completed the platform by introducing observability. Prometheus was deployed to collect metrics from nodes, the Kubernetes control plane, and application containers, while Grafana provided centralized visualization through version-controlled dashboards. AlertManager enabled proactive alerting via email notifications, ensuring full visibility and rapid response across the system.

Overall, this project transformed BeBouncy's infrastructure from manually managed services into a structured DevSecOps platform emphasizing automation, security, and observability. Deployments are now fully Git-driven, artifacts are systematically analyzed before promotion, secrets are centrally managed through Vault, and system behavior is continuously monitored with proactive alerting.

Perspectives. Future improvements could include AI-driven anomaly detection in Grafana for predictive monitoring, expansion to a multi-node or multi-cloud Kubernetes architecture to improve scalability and resilience, and adoption of a zero-trust model with mutual TLS between services. These enhancements would further strengthen reliability and support the evolution toward a fully self-healing DevSecOps platform.

Webography

[1] BeBouncy, "BeBouncy Platform," *bebouncy.com*. [Online]. Available: https://www.bebouncy.com/ and https://qa.bebouncy.com/. [Accessed: Feb. 10, 2025].

[2] Fibery, "Work Management Tool," *fibery.io*. [Online]. Available: https://fibery.io/. [Accessed: Feb. 14, 2025]. Slack Technologies, "Team Collaboration Platform," *slack.com*. [Online]. Available: https://slack.com/. [Accessed: Feb. 18, 2025].

[3] Docker, Inc., "Docker Documentation," *docs.docker.com*. [Online]. Available: https://docs.docker.com/. [Accessed: Feb. 22, 2025]. Docker, Inc., "node — Official Image," *Docker Hub*. [Online]. Available: https://hub.docker.com/_/node. [Accessed: Feb. 28, 2025].

[4] GitHub, Inc., "Working with the Container Registry," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry. [Accessed: Mar. 3, 2025].

[5] The Kubernetes Authors, "Kubernetes Documentation," *kubernetes.io*. [Online]. Available: https://kubernetes.io/docs/. [Accessed: Mar. 10, 2025].

[6] The Helm Authors, "Helm The Kubernetes Package Manager," *helm.sh*. [Online]. Available: https://helm.sh/docs/. [Accessed: Mar. 17, 2025].

[7] Argo CD Authors, "Argo CD Declarative GitOps CD for Kubernetes," *argo-cd.readthedocs.io*. [Online]. Available: https://argo-cd.readthedocs.io/. [Accessed: Mar. 21, 2025]. OpenGitOps, "OpenGitOps Principles," *opengitops.dev*. [Online]. Available: https://opengitops.dev/. [Accessed: Mar. 26, 2025].

[8] GitHub, Inc., "GitHub Actions Documentation," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/actions. [Accessed: Apr. 1, 2025]. Docker, Inc., "docker/build-push-action," *GitHub*. [Online]. Available: https://github.com/docker/build-push-action. [Accessed: Apr. 5, 2025].

[9] SonarSource, "SonarCloud Documentation," *docs.sonarcloud.io*. [Online]. Available: https://docs.sonarcloud.io/. [Accessed: Apr. 11, 2025].

[10] Aqua Security, "Trivy Vulnerability and Misconfiguration Scanner," *trivy.dev*. [Online]. Available: https://trivy.dev/. [Accessed: Apr. 14, 2025]. CycloneDX Project, "CycloneDX SBOM Standard," *cyclonedx.org*. [Online]. Available: https://cyclonedx.org/. [Accessed: Apr. 19, 2025].

[11] Bridgecrew, "Checkov Infrastructure as Code Static Analysis," *checkov.io*. [Online]. Available: https://www.checkov.io/. [Accessed: Apr. 21, 2025]. Center for Internet Security, "CIS Kubernetes Benchmark," *cisecurity.org*. [Online]. Available: https://www.cisecurity.org/benchmark/kubernetes. [Accessed: Apr. 23, 2025]. OWASP Foundation, "OWASP ZAP, Zed Attack Proxy," *zaproxy.org*. [Online]. Available: https://www.zaproxy.org/. [Accessed: Apr. 26, 2025].

[12] HashiCorp, "Vault Documentation," *developer.hashicorp.com*. [Online]. Available: https://developer.hashicorp.com/vault/docs. [Accessed: Apr. 28, 2025].

[13] Prometheus Authors, "Prometheus Documentation," *prometheus.io*. [Online]. Available: https://prometheus.io/docs/. [Accessed: May 4, 2025]. Prometheus Community, "kube-prometheus-stack Helm Chart," *GitHub*. [Online]. Available: https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack. [Accessed: May 8, 2025]. Prometheus Operator Authors, "Prometheus Operator Documentation," *prometheus-operator.dev*. [Online]. Available: https://prometheus-operator.dev/. [Accessed: May 10, 2025].

[14] Grafana Labs, "Grafana Documentation," *grafana.com*. [Online]. Available: https://grafana.com/docs/grafana/latest/. [Accessed: May 15, 2025]. rfmoz, "Node Exporter Full Dashboard," *Grafana Dashboards*. [Online]. Available: https://grafana.com/grafana/dashboards/1860-node-exporter-full/. [Accessed: May 20, 2025]. Prometheus Authors, "Alertmanager Documentation," *prometheus.io*. [Online]. Available: https://prometheus.io/docs/alerting/latest/alertmanager/. [Accessed: May 25, 2025].

Abstract

This report presents the progressive design and implementation of a DevSecOps platform for BeBouncy, an intelligent solution for multichannel management, analysis, and automation. The work is organised into three complementary sprint releases. The first release replaces the former manual PM2-based deployment model with a containerised and GitOps-driven architecture using Docker, Kubernetes, Helm, GitHub Actions, GHCR, and ArgoCD. The second release strengthens the delivery pipeline by integrating security controls across the software lifecycle, including static code analysis with SonarCloud, dependency and container image scanning with Trivy, Infrastructure-as-Code validation with Checkov, dynamic application testing with OWASP ZAP, and secure secret management through HashiCorp Vault and Kubernetes Secrets. The third release completes the platform with an observability layer based on Prometheus, Grafana, and AlertManager, enabling metric collection, cluster dashboarding, and operational alerting. The resulting solution improves reliability, security, traceability, and monitoring capabilities in the QA environment while moving the infrastructure closer to a production-like operating model.

Keywords : DevSecOps, Kubernetes, GitOps, CI/CD, Security, Observability

Résumé

Ce rapport présente la conception et la mise en œuvre progressive d'une plateforme DevSecOps pour BeBouncy, une solution intelligente de gestion, d'analyse et d'automatisation multicanale. Le travail est structuré autour de trois releases complémentaires. La première release transforme le déploiement manuel basé sur PM2 en une architecture conteneurisée et pilotée par GitOps, à travers Docker, Kubernetes, Helm, GitHub Actions, GHCR et ArgoCD. La deuxième release renforce la chaîne de livraison par l'intégration de contrôles de sécurité couvrant l'analyse statique du code avec SonarCloud, l'analyse des dépendances et des images avec Trivy, la validation Infrastructure-as-Code avec Checkov, les tests dynamiques avec OWASP ZAP, ainsi que la gestion sécurisée des secrets avec HashiCorp Vault et Kubernetes Secrets. La troisième release complète la plateforme par une couche d'observabilité basée sur Prometheus, Grafana et AlertManager, permettant la collecte des métriques, la visualisation de l'état du cluster et l'envoi d'alertes opérationnelles. L'ensemble de la solution améliore la fiabilité, la sécurité, la traçabilité et la capacité de supervision de l'environnement QA tout en rapprochant l'infrastructure d'un fonctionnement production-like.

Mots clés : DevSecOps, Kubernetes, GitOps, CI/CD, sécurité, observabilité